

Klander

A shared calendar for friend groups: project report

Ethan Jeknine

Portfolio project · October 2026 (updated after phase 11)

Live app: klander.vercel.app

Code: github.com/eejeknine/Klander

Built with React, Supabase, Gemini and Vercel, with Claude as AI development assistant

Contents

Summary.....	5
1. Analysis.....	6
1.1 Background and the problem.....	6
1.2 Stakeholders.....	6
1.3 Gathering requirements.....	6
1.4 Research into existing solutions.....	7
1.5 Objectives.....	7
1.6 Constraints.....	8
1.7 Choosing the technology.....	8
App type.....	9
Backend.....	9
AI for smart add.....	9
1.8 Development approach.....	9
1.9 Tools and AI assistance.....	10
1.10 Objectives for phases 6–11.....	11
2. Design.....	12
2.1 System architecture.....	12
2.2 Data model.....	13
2.3 Security and privacy design.....	14
Row Level Security (RLS).....	14
Privacy levels on events.....	14
Secrets.....	15
2.4 Key algorithms.....	15
Expanding repeating events across clock changes.....	15
Laying out overlapping events in a day column.....	15
Importing a calendar link.....	16
Smart add pipeline.....	16
2.5 User interface design.....	16
2.6 Server API design (Edge Functions and RPCs).....	17
2.7 Test strategy.....	17
2.8 Design for phases 6–11.....	17
New parts of the system.....	18
Data model.....	18
Security design for the new features.....	19
Key algorithms.....	20
Interface redesign (UI v2).....	20
3. Development.....	22
3.1 Phase 0: Planning and roadmap.....	22
3.2 Phase 1: Setup.....	22
What was built.....	22
My role.....	22
Problems and fixes.....	22
3.3 Phase 2: Your calendar.....	23
What was built.....	23
Problems and fixes.....	25
3.4 Phase 4: Friends (built before phase 3).....	25
What was built.....	25
My role.....	26
3.5 Phase 3: Import and sync.....	26

What was built.....	26
Problems and fixes.....	27
3.6 Phase 5: Smart add.....	27
What was built.....	27
My role.....	28
Problems and fixes.....	28
3.7 Version control and deployment.....	28
3.8 Phase 6: Themes and birthdays.....	29
3.9 Phase 7: Friends and plans.....	29
3.10 Phase 10 part 1: Notifications (brought forward).....	30
3.11 UI v2 and phase 8: Chat.....	31
3.12 Help and welcome tour.....	32
3.13 Phase 9: Memories and My Week.....	33
3.14 Phase 10 part 2: Shifts, family link and smarter editing.....	33
3.15 Phase 11: Insights.....	34
3.16 Add to Home Screen guide.....	35
3.17 Version control (phases 6–11).....	35
4. Testing.....	37
4.1 Database security tests.....	37
Phase 2: your own data.....	37
Phase 4: friends and privacy (Alice, Bob, Carl).....	37
Activity feed and calendar links.....	38
4.2 Unit tests: logic.....	38
Repeating events across the clock change.....	38
Calendar file (.ics) parser.....	39
Smart add: AI draft to event.....	39
4.3 Live server tests.....	39
4.4 Automated interface tests.....	40
4.5 Bugs found by testing.....	40
4.6 My real-world testing.....	41
4.7 Database privacy tests: phases 6–11.....	41
4.8 Logic tests: phases 6–11.....	42
4.9 Interface tests: phases 6–11.....	43
4.10 Bugs found in phases 6–11.....	43
5. Evaluation.....	44
5.1 Against the objectives.....	44
5.2 What went well.....	44
5.3 Limitations.....	45
5.4 Cost and environmental impact.....	45
5.5 What I learned.....	45
5.6 Objectives for phases 6–11.....	46
5.7 Reflection on phases 6–11.....	46
6. Future development.....	47
Appendix A: Database schema.....	48
Appendix B: Key code excerpts.....	59
B1. Expanding repeating events (src/lib/dates.js).....	59
B2. Overlap layout (src/lib/dates.js).....	59
B3. Friends' events with privacy applied (database function).....	60
B4. Activity trigger (database).....	60
B5. Live updates (src/lib/friends.js).....	61
B6. Reading calendar time zones (supabase/functions/_shared/ics.js).....	61

B7. Smart add prompt rules (supabase/functions/smart-add)..... 62

B8. Turning an AI draft into an event (src/lib/smart.js)..... 62

B9. Free-time finder (src/lib/freetime.js)..... 63

B10. Sleep after shifts (src/lib/sleep.js)..... 63

B11. Streaks (src/lib/insights.js)..... 63

B12. Hangout counts (database function)..... 64

Appendix C: Glossary..... 65

Summary

Klander is a free calendar app for groups of friends. Everyone has their own account and calendar. You can see your friends' weeks with privacy you control per event (full details, "Busy" only, or private), and add events by typing a sentence or taking a photo. It pulls in calendar links such as a uni timetable and publishes its own link for Apple, Google and Outlook. It installs on an iPhone from Safari without the App Store, and runs on free tiers for £0 a month.

Since the first version (phases 1–5) it has grown plans with invites, chat, photo memories, lock-screen notifications, help for new users and private stats. These are covered in Sections 1.10, 2.8, 3.8–3.17, 4.7–4.10 and 5.6–5.7.

Problem	Friends can't easily see when each other are free; plans get lost in group chats
Solution	A shared-by-default personal calendar with privacy levels, live updates and AI quick-add
Stack	React + Vite PWA · Supabase (Postgres, Auth, RLS, Realtime, Storage, Edge Functions, pg_cron) · Gemini · Vercel
Status	11 of 12 phases shipped and live; phase 12 (launch and portfolio) next
Testing	Phases 1–5: 67 recorded tests. Phases 6–11: 34 live database privacy checks, 269 automated logic checks and 12 interface journeys. All passing.
My role	Idea, requirements, every product decision, all service setup and configuration, deployment and real-world testing. Claude walked me through each stage and wrote most of the code.

1. Analysis

1.1 Background and the problem

I work a night shift as a warehouse operative (Monday to Friday, 23:30 to 08:30), I started a Data Science with AI degree at Leeds Beckett in September 2026, and I play basketball and volleyball. My friends all have their own mix of uni, work and sport. Trying to find a time when we are all free usually means a long group chat, people forgetting what they said, and plans that never happen.

Each of us already uses a calendar, but they are all separate. Google Calendar and Apple Calendar can share calendars, but you have to share each calendar with each person one at a time, and nobody does it. Tools like When2meet are good for one meeting, but you fill them in from scratch every time and they forget everything afterwards. My uni timetable lives in a separate system again.

The problem: there is no simple, free place where a group of friends can each keep their own calendar, see each other's weeks at a glance with sensible privacy, and add plans quickly from the messages, posters and screenshots where plans actually come from.

1.2 Stakeholders

Stakeholder	Who they are	What they need from Klander
Me (primary user and developer)	Student and night-shift worker in Leeds, mainly uses an iPhone	One calendar for shifts, lectures and sport; see friends' free time; a real portfolio project
My friends (5–6 people)	Students and workers with irregular weeks	Quick sign-up, see each other's weeks, control what others can see, add plans quickly
My mum	Uses Apple Calendar for everything	See my plans in Apple Calendar without installing anything new
Future employers / reviewers	People reading my portfolio	Evidence of a complete, secure, well-tested full-stack app with clear reasoning

1.3 Gathering requirements

I started by writing out everything I wanted the app to do in plain language, then turned it into a list of features and grouped them into phases. The first request was: *"a calendar app that my friends can use and make their own accounts, where we can see each other's week, and where I can take a picture or type some text and turn it into an event."* From there I asked for more ideas and picked the ones that solved real problems for my group.

As the project went on, testing it myself led to new requirements, which is why the plan changed between phases:

- **Calendar links:** my uni gives me a calendar link for my timetable, so the app needed to read those links and stay in sync with Apple, Google and Outlook.
- **Friends before sync:** I swapped the order of two phases so that the friends feature (the main reason for the app) came earlier.

- **Testing before inviting friends:** I didn't want to send the app to friends until I was sure the privacy and friends features worked, so I tested with a second account first.
- **Later ideas** (themes, birthdays, chat, photo memories, notifications) came from using the app. They were built in phases 6–11 (Section 1.10).
- **Notifications brought forward:** I asked whether notifications worked; they didn't, so the first half of phase 10 was built before phase 8.
- **Two-way Apple sync dropped:** it needed my Apple app-specific password, so I chose not to build it.

1.4 Research into existing solutions

I compared the tools my friends and family already use. This is a summary of how each one fits our problem, based on using them, rather than a full feature audit.

Existing solution	Strengths	Why it doesn't solve our problem
Google Calendar	Free, reliable, good on Android, can subscribe to links	Sharing is per calendar and per person; no friend list or "see everyone's week" view; no privacy levels designed for friend groups
Apple Calendar (iCloud)	Built into iPhones; my mum already uses it	Shared calendars are awkward to set up for a group; no easy way to compare several people's free time
When2meet-style polls	Great for finding one meeting time	Starts from scratch every time and does not know your real calendar
Group chats	Where plans actually start	Plans get lost in the scroll; no calendar, no reminders
Social calendar apps	Built around friends	Usually need an App Store install and an account for everyone, and may not read uni timetable links

What I took from this: Klander should (1) be a real personal calendar first, (2) layer friends on top with privacy levels per event, (3) read and publish standard calendar links so it works alongside Apple, Google and Outlook rather than replacing them, and (4) make adding events faster than typing them out.

1.5 Objectives

These are the measurable objectives I set for the first five phases. Section 5 evaluates each one, and Section 1.10 adds objectives for phases 6–11.

#	Objective	How I will know it is met
1	People can create their own account with email or Google and set a unique username, display name and colour	A new user can sign up, confirm, and reach the calendar in under 2 minutes; usernames are unique
2	A full personal calendar: day, week, month and list views; create, edit and delete events	Every view displays the same events correctly at phone width (390 px)

#	Objective	How I will know it is met
3	Repeating events, including overnight shifts, that stay at the right local time when the clocks change	A weekday 23:30–08:30 shift still shows at 23:30 after 25 October 2026
4	Delete a single occurrence of a repeating event, or the whole series	Skipped date disappears; other repeats remain
5	Categories with colours, and a visibility setting on every event (Friends, Busy only, Private)	Category colours show in all views; visibility is saved per event
6	Installable on iPhone for free and usable offline (read-only)	Add to Home Screen opens full-screen; last calendar shows with no signal
7	Add friends by username or invite link, and see friends' events in their colour	Two test accounts see each other's events after accepting
8	Privacy is enforced by the database, not just hidden in the app	A stranger sees 0 events; Busy shows only "Busy"; Private never leaves the owner's account
9	Friends' changes appear live, with an activity feed	A change on one device appears on the other within seconds
10	Import calendar links (.ics / webcal) that refresh automatically, and publish a link other apps can subscribe to	Linked calendar refreshes at least every 3 hours; Apple Calendar can subscribe to Klander
11	Turn a typed sentence, photo or screenshot into events that I can review before saving	Common sentences produce the correct date and time; nothing saves without confirmation
12	Running cost of £0 per month for my friend group	All services on free tiers

1.6 Constraints

- **Cost:** I am a student, so everything has to run on free tiers. Paying £99 a year for an Apple developer account was not an option.
- **iPhone without the App Store:** most of my friends use iPhones. Free sideloaded apps expire every 7 days, so the app had to be a Progressive Web App (PWA) installed from Safari.
- **Time:** I work nights, so I could only work on the project in short sessions. The work was split into phases with a clear "done when" for each.
- **Privacy:** the app stores friends' personal schedules, so data protection had to be designed in from the start, not added later.
- **My experience:** I study Computer Science but had not built a full-stack app with authentication, a cloud database and serverless functions before, so I needed guidance (see 1.9).

1.7 Choosing the technology

I compared options for each part of the system before starting.

App type

Option	Pros	Cons	Decision
Native iOS app (Swift)	Best performance, widgets, App Store	£99/year Apple fee; iPhone only; free builds expire every 7 days	Rejected
Progressive Web App (React + Vite)	Free; works on iPhone, Android and desktop; installs to Home Screen; one codebase	No home-screen widgets; iPhone push needs iOS 16.4+	Chosen
Plain HTML/JS (like my other tools)	Simple, no build step	Hard to manage at this size	Rejected

Backend

Option	Pros	Cons	Decision
Supabase	Postgres (SQL I can learn from), auth, realtime, storage and functions in one free project; Row Level Security for privacy	Free projects pause after a week of no use	Chosen
Firebase	Popular, generous free tier	NoSQL makes the friends/privacy queries harder; less useful for my SQL learning	Rejected
Own server	Full control	Costs money, more to secure and maintain	Rejected

AI for smart add

Option	Cost for ~7 people	Notes	Decision
Gemini (free tier)	£0	Reads text and images; free tier may use data to improve Google's models	Chosen for now
Claude Haiku	~£1/month	Needs prepaid credit	Rejected for now
Gemini (paid)	~35p/month	Data not used for training	Planned before launch

Hosting is on **Vercel** (free, deploys automatically when I push to **GitHub**).

1.8 Development approach

I used an **iterative, phased approach**. Each phase had a list of features and a "done when" test. At the end of each phase I tested it myself, fixed what I found, pushed it live and then decided the next phase, sometimes changing the order based on what I learned. I tracked the plan in an interactive roadmap page.

Phase	Content	Status
1. Setup	Supabase, PWA, hosting, accounts	Done
2. Your calendar	Views, events, repeats, categories, settings	Done

Phase	Content	Status
3. Import & sync	Calendar links in, subscribe link out	Done
4. Friends	Requests, invites, overlay, privacy, live updates, activity	Done
5. Smart add	AI from text, photos and timetables	Done
6. Themes & birthdays	Themes, contrast guard, birthdays, UK dates	Done
7. Friends & plans	Free-time finder, invites, polls, close friends, groups	Done
8. Chat	DMs, groups, plan chats, photos, block and report	Done
9. Memories	Event photos, My Week story	Done
10. Notifications & sync	Push, reminders, shifts and sleep, family link, morning brief	Done (two-way Apple sync skipped)
11. Insights	Stats, streaks, hangouts, Wrapped	Done
12. Launch & portfolio	Privacy, data export and deletion, landing page, demo	Next

1.9 Tools and AI assistance

I built Klander with **Claude** (Anthropic's AI assistant) as my development partner, and I want to be clear about who did what.

What Claude did: walked me through each stage step by step, explained the options and trade-offs, and wrote most of the code, especially the difficult parts: the database security rules, the repeating-event logic, the calendar file (.ics) parser, the serverless functions and the AI prompt. It also ran automated tests on the code it wrote.

What I did:

- Came up with the idea, the name and the features, and decided what to build in each phase and in what order.
- Made the product decisions: privacy levels, PWA rather than native, free tiers, which AI provider, and which features to cut or add.
- Set up and configured every service myself: the Supabase project settings, Google Cloud sign-in (OAuth client, redirect URLs), the Gemini API key, the GitHub repository and the Vercel deployment with its environment variables.
- Ran the app on my Mac and iPhone, tested every feature with real use and a second account, found problems and reported them so they could be fixed (Section 3 lists them).
- Pushed each version live with Git.

Using an AI assistant is now a normal part of professional software development. The skill I practised was directing it well: deciding what to build, checking the output, testing it properly, and understanding how the system works well enough to explain it. The companion document "How Klander works" is my own study guide for that.

1.10 Objectives for phases 6–11

After phase 5 my friends and I had used the app for a while, so the next objectives came from real gaps: we could see each other's weeks but still made plans in the group chat, nobody got told when something changed, and my night shifts made "free" hard to judge. Section 5.6 evaluates these.

#	Objective	How I will know it is met
13	Make it feel like ours: themes, birthdays	Eight themes plus custom ones that always stay readable; friends' birthdays show automatically, with age only if they allow it
14	Find a time everyone is free without anyone sharing details	Free-time finder uses busy times only; private titles never leave the database
15	Turn a free time into a plan with invites and answers	Invite friends, they answer Going / Maybe / Can't, the host sees who is coming
16	Finer privacy	A "Close friends" level, and surprise events hidden from chosen people
17	Chat inside the app, tied to plans	Direct, group and per-plan chats; only members can read them; blocking works
18	Photo memories and a weekly story	Photos on events; a "My Week" story that leaves out private things by default
19	Notifications on the lock screen	Invites, friends' changes, messages and reminders arrive even with the app closed; quiet hours respected
20	Work around night shifts	Sleep after a shift is treated as busy, for me and for the free-time finder
21	Share my week with family without an app	A family calendar link that never shows private events
22	Fun, private stats	Hours by category, streaks, who I made plans with, an end-of-term "Wrapped"
23	Easy for new people	A welcome tour, a searchable help section and a step-by-step "Add to Home Screen" guide
24	Still £0 a month	All new features stay on free tiers

2. Design

2.1 System architecture

Klander has three main parts: the **app** that runs on the phone, **Supabase**, which holds the data and runs the server-side code, and a few **outside services** (Gemini for AI, plus other people's calendar links). The phone talks to Supabase directly. Anything that needs a secret key, or must not be trusted to the phone, runs in an **Edge Function** on Supabase's servers.

Klander system architecture

Everything runs on free tiers. The phone talks to Supabase directly; secret work happens in Edge Functions.

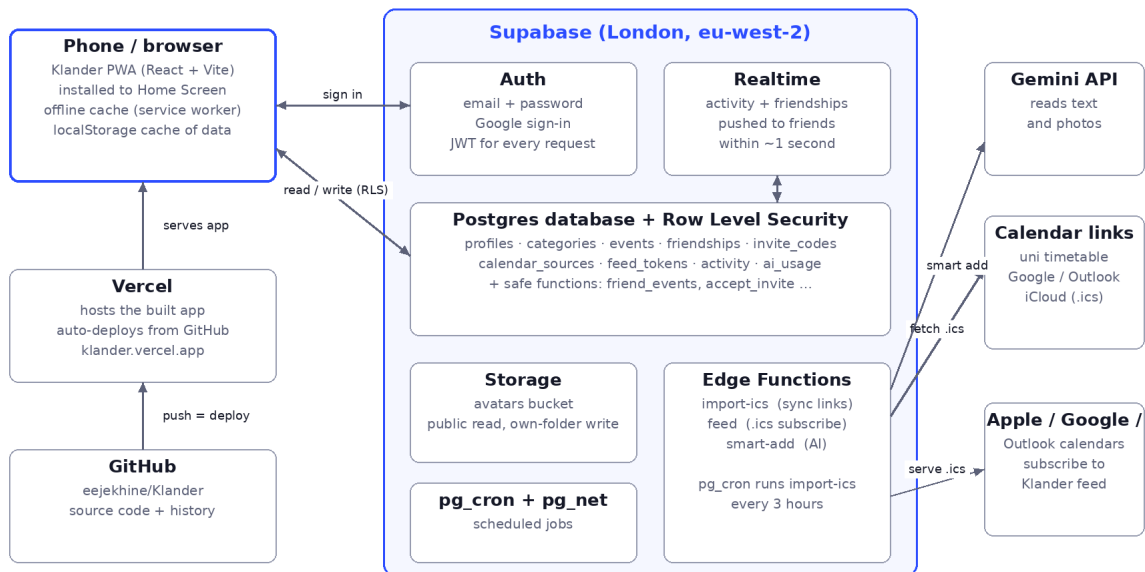


Figure 1: Klander system architecture (phases 1–5; see Figure 3 for now)

Part	Technology	Job
Front end	React 18 + Vite, vite-plugin-pwa	All screens; installable; caches the app for offline use
Hosting	Vercel	Serves the built app at klander.vercel.app; redeploys on every Git push
Accounts	Supabase Auth	Email/password and Google sign-in; gives the app a signed token (JWT) that proves who you are
Database	Supabase Postgres + Row Level Security	Stores everything; RLS rules decide which rows each person can read or change
Live updates	Supabase Realtime	Pushes friends' activity and friend requests to open apps
Files	Supabase Storage	Profile photos
Server code	Supabase Edge Functions (Deno/TypeScript)	import-ics (sync calendar links), feed (publish .ics), smart-add (AI)
Scheduling	pg_cron + pg_net	Calls import-ics every 3 hours

Part	Technology	Job
AI	Google Gemini API	Reads sentences and photos and returns structured event data

2.2 Data model

I designed the database around **one row per real thing**: people (profiles), events, categories, friendships and linked calendars. Events store a repeat rule instead of one row per repeat, so a weekly lecture for a whole term is a single row.

Klander database (entity relationship diagram)

PK = primary key, FK = foreign key. Every table has Row Level Security switched on.

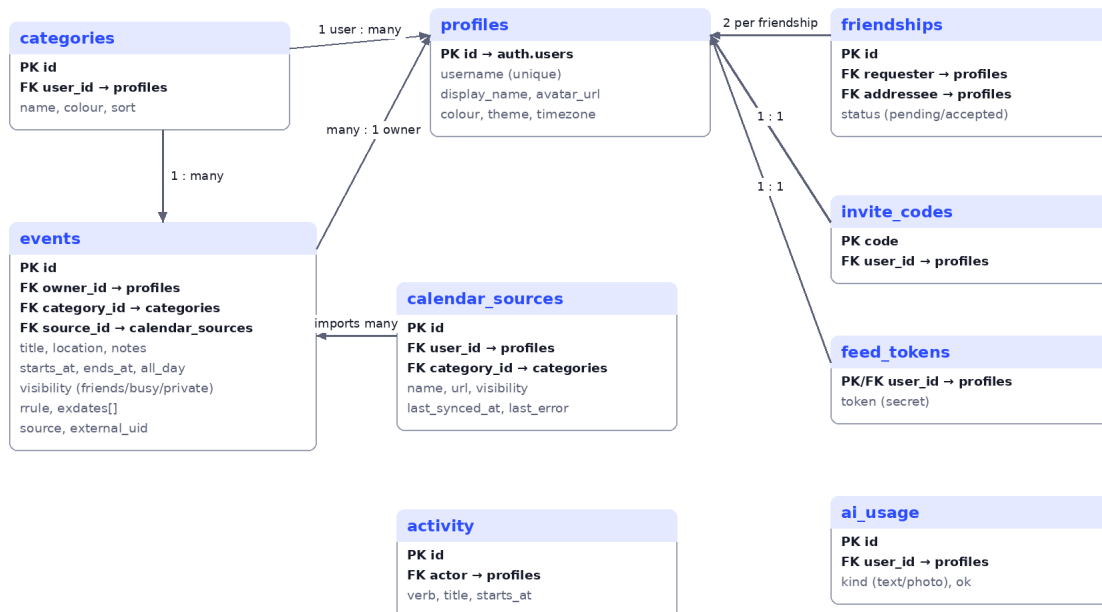


Figure 2: Entity relationship diagram

Table	Purpose	Key design decisions
profiles	One per user	Created automatically by a trigger on sign-up; username is case-insensitive (citext) and unique; colour is checked to be a hex code
categories	Work, Uni, Sport, Social, Personal	Five defaults created on sign-up; each user owns their own
events	Every event	starts_at/ends_at stored in UTC; rrule holds the repeat rule; exdates lists skipped dates; visibility is friends/busy/private; source records how it was made
friendships	Requests and friends	One row per pair; a unique index on (least, greatest) stops duplicate pairs; only the status column can be updated
invite_codes	Personal invite links	Random 18-character code; only the owner can read it
calendar_sources	Linked .ics calendars	Stores the link, last sync time, last error and the category/privacy to apply

Table	Purpose	Key design decisions
feed_tokens	Secret subscribe link	36-character random token; can be reset
activity	Activity feed	Written only by a database trigger, never by the app; privacy is applied when it's written
ai_usage	Smart-add limit	One row per AI request so each user is limited to 30 per day

2.3 Security and privacy design

Because the app holds friends' schedules, I made the **database itself** responsible for privacy rather than the app. If the app had a bug, or someone used the API directly, they still could not see data they shouldn't.

Row Level Security (RLS)

Every table has RLS switched on. By default nobody can see anything; each policy then opens up exactly one thing:

Table	Who can do what
events	You can read and write only your own events. Friends never query this table directly.
profiles	Any signed-in user can read basic profiles (so you can search usernames); you can only update your own.
friendships	You can see rows you are part of; only the person who received a request can accept it; either person can remove it; nobody can insert directly.
activity	Visible only to friends of the person who did it.
invite_codes, feed_tokens, calendar_sources, ai_usage	Owner only.
storage (avatars)	Anyone can view photos; you can only upload into a folder named after your own user id.

Privacy levels on events

Level	What friends see
Friends	Title, time and location (never notes)
Busy only	A block labelled "Busy" at that time, with no title or location
Private	Nothing at all

Friends' events come from one database function, `friend_events()`. It runs with elevated rights (SECURITY DEFINER) but always filters by the signed-in user, joins only accepted friendships, drops private events and rewrites busy ones before anything leaves the database. Friend requests and invite links also go through functions (`send_friend_request`, `accept_invite`), so the rules (no duplicates, no friending yourself, invites only work with a valid code) can't be bypassed.

Secrets

- The app only contains the Supabase **publishable** key, which is safe to be public because RLS protects the data.
- The **service role key** and the **Gemini API key** exist only inside Edge Functions as server-side secrets.
- The scheduled sync proves itself with a random secret stored in a private schema that the API cannot reach.
- The import function refuses links to private network addresses, so it can't be tricked into fetching internal resources.

2.4 Key algorithms

Expanding repeating events across clock changes

A repeating event is stored once, with a rule such as `FREQ=WEEKLY;BYDAY=MO,TU,WE,TH,FR`. To show a week, the app expands the rule into actual dates. The tricky part is British Summer Time: my 23:30 shift is 22:30 UTC in October but 23:30 UTC in November. If the expansion was done in UTC, the shift would move to 22:30 after the clocks change. The solution uses "**floating time**": the local wall-clock time is converted as if it were UTC, the repeats are generated, then each result is read back as local time.

```
FUNCTION expandEvents(events, from, to):
  results = []
  FOR each event IN events:
    duration = event.end - event.start
    IF event has no repeat rule:
      IF event overlaps [from, to): add event
      CONTINUE
    rule = parse(event.rrule) with start = toFloating(event.start) // local 23:30 ->
    "23:30Z"
    FOR each f IN rule.between(toFloating(from - duration), toFloating(to)):
      start = fromFloating(f) // "23:30Z" -> local
      23:30
      IF start IN event.exdates: CONTINUE // skipped occurrence
      end = start + duration
      IF start < to AND end > from: add copy of event with start, end
    RETURN results sorted by start
```

Laying out overlapping events in a day column

Events that overlap in time have to sit side by side. Events are sorted by start time and grouped into **clusters** of events that overlap each other. Within a cluster, each event goes into the first column that is free when it starts (a greedy interval-colouring approach). Each event's width is then 1 divided by the number of columns in its cluster.

```
sort events by start, longest first
FOR each event:
  IF event.start >= end of current cluster: finish cluster
  add event to cluster; cluster.end = max(cluster.end, event.end)
finish cluster:
  columns = [] // end time of the last event in each column
  FOR each event in cluster:
    c = first column whose end <= event.start, else a new column
```

```
columns[c] = event.end; event.col = c
every event in cluster gets width = 1 / number of columns
```

Importing a calendar link

- 1. Turn `webcal://` into `https://`, refuse private addresses, then download (25-second timeout, 8 MB limit).
- 2. Parse the .ics file and register any time zones it defines.
- 3. For each event: work out the real start/end instant. If the time zone is missing or a Windows name such as "GMT Standard Time", fall back to the user's zone (Europe/London).
- 4. Keep repeat rules as rules. Moved occurrences become their own event, and the original date is added to the series' skipped dates. Cancelled events are dropped.
- 5. Upsert by (source, UID) so re-syncing updates rather than duplicates, then delete events that have disappeared from the feed.
- 6. Record the sync time and count, and add one "updated their linked calendar" activity row (not one per event).

Smart add pipeline

- 1. The phone shrinks the photo to at most 1600 px (JPEG) and sends it with the text, the current local date and time, and the time zone.
- 2. The Edge Function checks the user is signed in and under 30 requests in the last 24 hours.
- 3. It sends Gemini a set of rules (UK dates, resolve "thurs" to the next Thursday, 24-hour times, overnight shifts end the next day, never invent details) plus a strict JSON schema the reply must follow.
- 4. If a model is unavailable or rate-limited, it tries the next model in a list.
- 5. The reply is validated (real dates, real times, allowed repeat values) and returned as drafts. Nothing is saved until the user reviews and confirms.

2.5 User interface design

The UI is designed **phone-first** (390 px wide) because that's how my friends will use it.

- **Navigation:** a header with the month, Today, back/forward and your avatar; a Day/Week/Month/List switch; and a strip of people (You plus each friend) you can tap to show or hide.
- **Adding:** a blue + button for a manual event and a "Smart add" button for AI.
- **Editing:** bottom sheets rather than new pages, so you never lose your place.
- **Colour:** each person has a colour, and your own events use category colours. Friends' events have a dotted edge, and busy blocks are striped so you can tell them apart at a glance.
- **Accessibility:** 16 px inputs (stops iPhone zooming), visible keyboard focus, labels on every control, reduced-motion support, and both light and dark themes built from the same colour tokens.

Token	Light	Dark	Use
--bg	#F3F4F8	#0F111A	Page background
--surface	FFFFFF	#181B27	Cards and sheets

Token	Light	Dark	Use
--ink	#151827	#ECEE6	Main text
--accent	#2B4CFF	#7D93FF	Buttons, today, links
--now	#E0477A	#FF7AA6	Current-time line
Fonts	Unbounded (headings), DM Sans (text)		Loaded from Google Fonts

2.6 Server API design (Edge Functions and RPCs)

Endpoint	Called by	Input → output	Auth
rpc send_friend_request	App	username → sent / accepted / already_sent / not_found / self	Signed-in user
rpc accept_invite	App (invite link)	code → friend's username or invalid	Signed-in user
rpc friend_events	App	(none) → friends' events with privacy applied	Signed-in user
rpc my_invite_code / my_feed_token	App	(reset?) → code / token	Signed-in user
POST /functions/v1/import-ics	App or schedule	{source_id} → {count, added, removed}	User token, or cron secret
GET /functions/v1/feed? token=	Apple/Google/ Outlook	token → .ics file	Secret token in URL
POST /functions/v1/smart-add	App	{text, image, now, tz} → {events[], message, remaining}	User token + daily limit

2.7 Test strategy

- **Database security tests:** SQL scripts that pretend to be different users (by setting the signed-in user inside a transaction) and check what each can see and change. They roll back afterwards, so no test data is left behind.
- **Unit tests** for the logic that is easy to get wrong: repeat expansion across the clock change, overlap layout, the .ics parser, and converting AI drafts into events.
- **Live tests** of the deployed Edge Functions, using a test account and a real public calendar (UK bank holidays from gov.uk).
- **Automated UI tests** in a headless browser at iPhone size (390 × 844), with screenshots of every screen in light and dark mode.
- **Real-world testing** by me on my Mac and iPhone, including a second account to act as a friend.

2.8 Design for phases 6–11

The same rules from phases 1–5 still apply: privacy is enforced by the database, anything needing a secret runs in an Edge Function, and the phone never gets more data than the person is allowed to see. Figure 3 shows the system now.

Klander system architecture (phase 11)

Everything runs on free tiers. The phone talks to Supabase directly; secret work happens in Edge Functions.

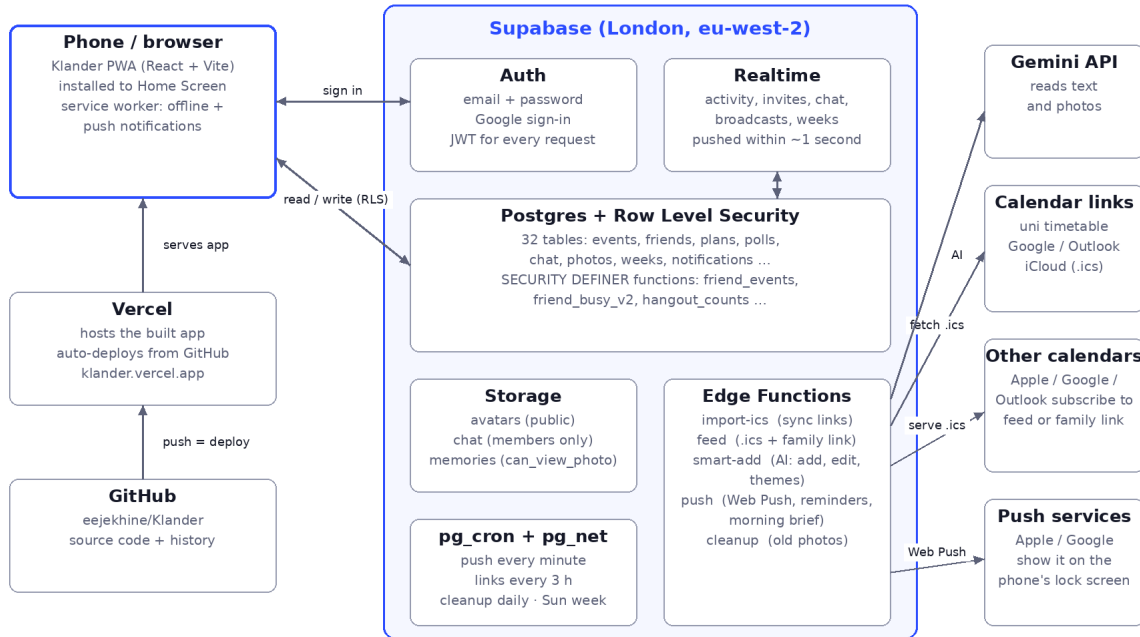


Figure 3: Klander architecture after phase 11

New parts of the system

Part	Job
push Edge Function	Creates its own VAPID key pair on first run (the private half never leaves the server), sends Web Push notifications, makes reminders and morning briefs, skips quiet hours and sleep after shifts, and removes phones that have uninstalled
cleanup Edge Function	Every night, deletes chat photos from deleted messages, chat photos older than 6 months, uploads that were never sent, and memory photos whose event is gone
Storage buckets	"chat" (members of that chat only) and "memories" (only people allowed to see that event or week), both private
pg_cron jobs	push every minute; calendar links every 3 hours; cleanup daily at 03:17; "your week is ready" on Sunday evenings
smart-add modes	As well as adding events, it now designs themes from a description and turns "move gym to 7" into a change you approve
feed (family link)	A second secret link for family: Friends events in full, Close and Busy-only as "Busy", Private never, notes never

Data model

17 tables were added (32 in total, all with Row Level Security). Figure 4 groups them by feature.

Tables added in phases 6-11, grouped by feature

Every table has Row Level Security. Arrows show the main links back to profiles and events.

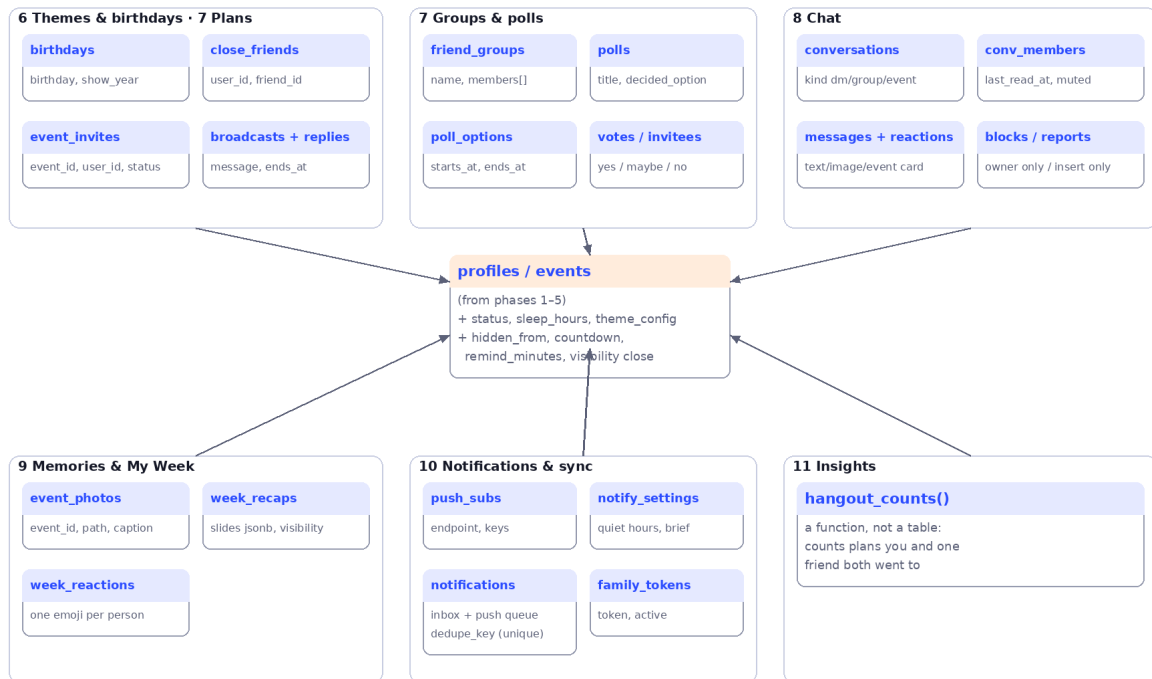


Figure 4: Tables added in phases 6-11

Security design for the new features

Feature	Rule (enforced in the database)
Close friends	Your list is private to you. Close events show in full only to people on it; everyone else gets "Busy". The activity feed never includes a close event's title.
Surprise events	<code>hidden_from</code> lists people who must not see the event at all, and they can't be invited to it.
Invites	You can only invite friends, only to your own events, and guests can only change their own answer (a column grant allows nothing else).
Free-time finder	<code>friend_busy_v2()</code> returns start and end times only: no titles, places or notes. It also tells the app which blocks are shifts, so friends' sleep counts as busy.
Chat	Everything is gated by "is this person a member?". You can only send as yourself, DMs are friends only, and blocking stops messages both ways.
Photos	<code>can_view_photo()</code> allows your own photos, photos of events you can see, or photos in a friend's posted week you're allowed to watch. Photos you hide in My Week are saved without their file path, so they can't be loaded.
Notifications	Only written by database triggers, never by the app. You can read your own and mark them read, nothing else.
Hangout stats	<code>hangout_counts()</code> only ever counts plans involving the caller and one friend, skips anyone blocked, and returns nothing to a stranger.

Key algorithms

Free-time finder. For each person, turn their events (with repeats expanded) into busy intervals, merge overlaps, then walk through each day between the chosen hours looking for gaps long enough for everyone. It also returns "everyone except one person" slots, which turned out to be the most useful.

```
FOR each day in the next N days:
  window = [day at startHour, day at endHour], starting no earlier than now (rounded up
  to :00/:30)
  FOR each person: free[person] = gaps in window not covered by merge(busy[person])
  everyone = intersect(free of all people)      -> keep gaps >= duration
  FOR each person p:
    allBut[p] = intersect(free of everyone except p) minus everyone -> keep gaps >=
  duration
RETURN everyone, allBut (best first)
```

Sleep after shifts. If a category is marked "These are shifts", every occurrence of an event in it gets a "Sleep" block from its end for my chosen number of hours (7 by default). Because it's calculated from each occurrence, it stays right across the clock change.

Reminders without duplicates. Every minute, the push function works out which reminders are due (the event's own setting, then its category's, then my default), and inserts each one with a `dedupe_key` like `rem:<event>:<occurrence time>`. The key is unique, so running twice can never send twice.

My Week. Built on the phone from my events and photos for that week. Private and Busy-only events are left out, photos from them are hidden by default, and only events marked Friends or Close can appear as text.

Streaks. For each category (except shifts), collect the weeks that had at least one event. Count back from this week. If this week hasn't had one yet, start from last week instead, so a streak isn't "broken" on a Monday morning.

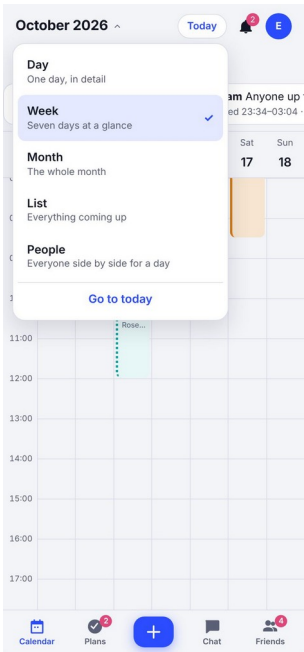
Interface redesign (UI v2)

Testing on my phone, the controls at the top felt crowded and changing week needed tiny arrow buttons. The redesign:

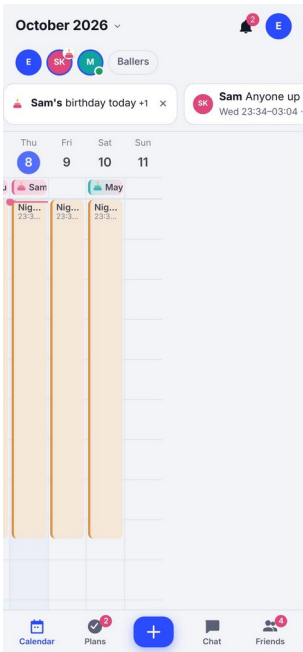
- **Swipe** left and right to change week, day or month.
- **A bottom tab bar:** Calendar · Plans · + · Chat · Friends, all in thumb reach.
- **The month name is the menu** for Day, Week, Month, List, People, Go to today, Insights and Help.
- **One row of avatars** to show or hide people, and **one row of cards** for birthdays, countdowns, friends' weeks and "add to Home Screen".



UI v2 week view



Title menu



Swiping weeks

3. Development

I built Klander in five phases. For each phase I describe what was built, who did what, the problems we hit and how they were fixed. All code is in my GitHub repository (eejekhhine/Klander), with one commit per phase.

3.1 Phase 0: Planning and roadmap

- I wrote the feature list and asked for more ideas, then picked the ones I wanted: friends' weeks, photo/text to event, invites and RSVPs, privacy per event, groups, recurring events, timetable import, travel time and more.
- We researched real costs. For 5–6 friends everything runs on free tiers (£0/month). The only paid option is AI, at roughly 35p to £1 a month if I switch to a paid tier.
- I asked for the roadmap as an **interactive HTML page** rather than a document, with tickable checklists per phase. I also renamed the app from "Cylinderr" to **Klander**.
- I chose the build approach: a new Supabase project in London, code in a folder on my Mac, React + Vite, and phases 1–2 first.

3.2 Phase 1: Setup

What was built

- A new Supabase project ([klander](#), eu-west-2 London region).
- A React + Vite project with [vite-plugin-pwa](#): web manifest, service worker, app icons (a calendar with a "K") and the iPhone home-screen meta tags.
- A Git repository in [~/Sites/klander](#) with the first commit, pushed to GitHub and deployed on Vercel at [klander.vercel.app](#).

My role

- Installed Node.js on my Mac when `node -v` showed it was missing, and ran the app locally with `npm install` and `npm run dev`.
- Created the GitHub repository, pushed the code and set up Vercel, including the two environment variables (Supabase URL and publishable key).
- Configured Supabase Authentication: Site URL and Redirect URLs so email links come back to the app.
- Set up Google sign-in end to end: created a Google Cloud project, the OAuth client and client secret, added the authorised JavaScript origins and redirect URI, and enabled the Google provider in Supabase.

Problems and fixes

Problem	Cause	Fix
zsh: command not found: node	Node.js wasn't installed on my Mac	Installed the LTS version from nodejs.org and reopened Terminal
Confirmation email link didn't open the app	Supabase's Site URL was still the default	The account was confirmed anyway; I set the Site URL and Redirect URLs in Supabase

Problem	Cause	Fix
<code>git push</code> : repository not found	The GitHub repo hadn't been created yet	Created the empty repo on GitHub, then pushed again
Pasted the Vercel <i>setup</i> URL instead of the app URL	The project hadn't been deployed yet	Clicked Create Project; the app went live at <code>klander.vercel.app</code>
"Continue with Google" didn't work	The Google provider wasn't enabled	Created the OAuth client in Google Cloud and enabled it in Supabase
Google: Error 400 <code>redirect_uri_mismatch</code>	The Supabase callback URL wasn't listed in Google Cloud	Added <code>https://...supabase.co/auth/v1/callback</code> as an authorised redirect URI
GitHub said "This repository moved"	I renamed the repo to Klander with a capital K	Pushes still worked; updated the remote with <code>git remote set-url</code>

3.3 Phase 2: Your calendar

What was built

- **Database:** profiles, categories, events and the avatars storage bucket, all with RLS. A trigger creates a profile and five default categories when someone signs up.
- **Accounts:** sign in, create account, forgot password, set new password, Google sign-in, and a first-time setup screen for username, display name and colour.
- **Views:** week (time grid with a "now" line), day, month (up to three events per day plus "+N more") and a 60-day list.
- **Events:** a bottom-sheet editor with title, all-day, start/end, repeat, category, visibility, location and notes. An end time earlier than the start automatically moves to the next day, which handles night shifts.
- **Repeats:** daily, weekdays, weekly, fortnightly, monthly and yearly, with an optional end date. You can delete one occurrence or the whole series.
- **Settings:** profile photo (cropped and resized on the phone before upload), name, username, colour, theme (match phone / light / dark), category editor, install instructions and sign out.
- **Offline:** the last loaded profile, categories and events are cached so the calendar still shows without signal.

Klander.

Your calendar, and your friends' weeks, in one place.

Sign in

Create account

Email

Password

Sign in

Forgot password?

or

Continue with Google

Sign in

Klander.

Set up your profile

Friends will find you by your username.

Username

ethan_j

Display name

Ethan Jekhine

Your colour (friends see your events in this)

Start using Klander

First-time setup

Cancel

New event

Save

Repeat

Every weekday (Mon–Fri)

Repeat until (optional)

mm/dd/yyyy

CATEGORY

None

Work

Uni

Sport

Social

Personal

WHO CAN SEE IT

Friends

Busy only

Private

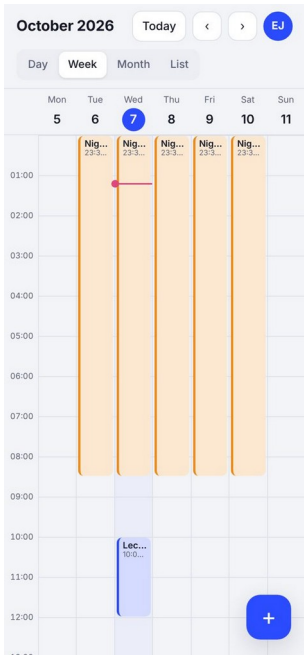
Friends see the title, time and place. Friends arrive in the next update.

Location

e.g. The Edge, Leeds Beckett

Notes

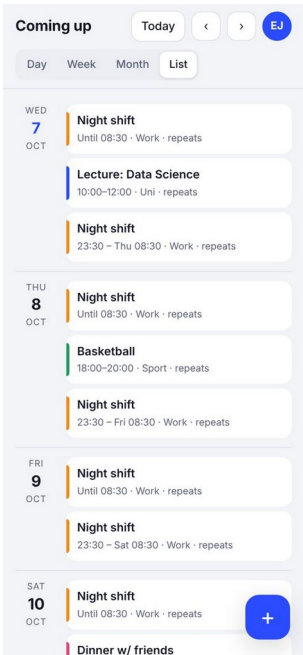
Event editor



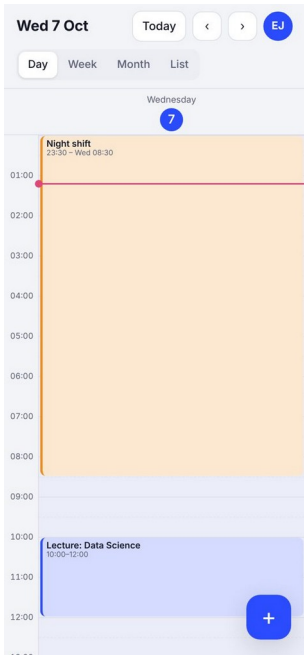
Week view



Month view



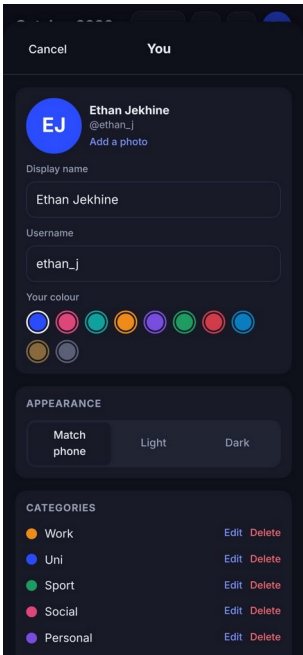
List view



Day view



Dark mode



Settings

Problems and fixes

Problem	Fix
Event titles were vertically centred in the week view (buttons centre their content by default)	Changed the event blocks to a top-aligned flex column
The visibility buttons wrapped onto two lines on a phone	Shortened the labels to "Friends", "Busy only" and "Private"
In the list view, an overnight shift on its second day showed "23:30 – Wed 08:30", which was confusing	Shows "Until 08:30" when the event started the day before
Repeat end dates were being shifted by the time-zone conversion	Stored the end date in the same "floating" format as the start, and verified it with a unit test

3.4 Phase 4: Friends (built before phase 3)

I asked whether the friends feature could be added before putting the app online. It could, because it lives in the database and app code, so I moved it earlier. The main reason for the app is seeing friends' weeks.

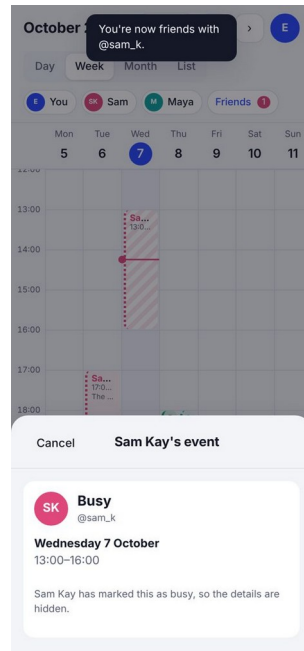
What was built

- **Database:** friendships, invite_codes and the functions send_friend_request, accept_invite, my_invite_code and friend_events. Friendships can only be created through these functions.
- **Friends sheet:** add by @username, a "Share my invite link" button (uses the phone's share menu), accept or decline requests, cancel sent requests, remove friends, and show or hide each friend.
- **Invite links:** opening `klander.vercel.app/?invite=CODE` remembers the code until you are signed in, then makes you friends automatically and shows a confirmation.

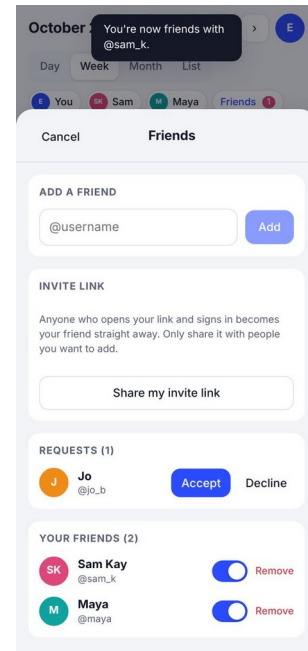
- **Calendar overlay:** friends' events in their colour with their first name. Busy blocks are striped, and tapping a friend's event opens a read-only view.
- **Live updates and activity feed** (finished in the same round as phase 3): a database trigger writes privacy-safe activity rows; Realtime pushes them to friends within about a second; the friends sheet shows "Sam added Gym · Tue 17:00"; a red badge counts unseen activity.



Friends on the week



A friend's busy block



Friends sheet

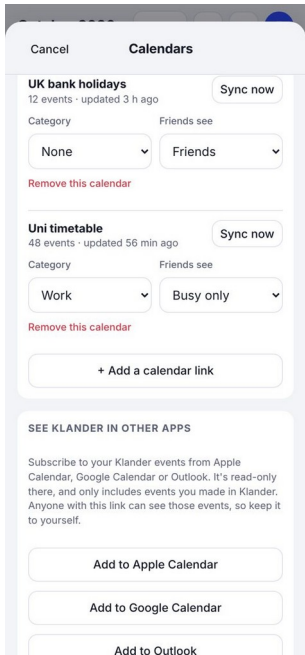
My role

- Decided the privacy model (Friends / Busy only / Private) and that invite links should make you friends instantly.
- Insisted on testing before sending it to friends: I created a second account with a "+test" email address in a private window and checked live updates, busy blocks and private events between the two accounts.

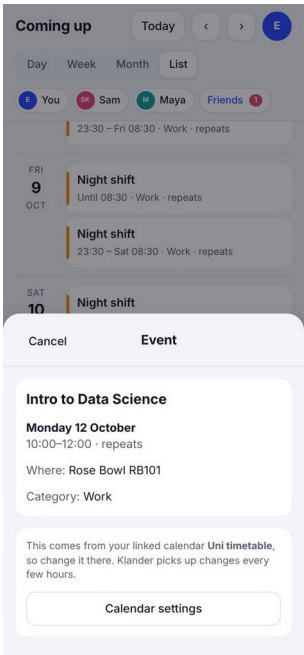
3.5 Phase 3: Import and sync

What was built

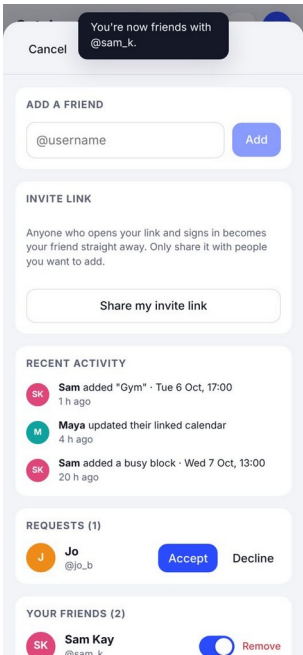
- **Linked calendars:** paste any .ics or webcal link (uni timetable, Google "secret iCal address", Outlook published calendar, iCloud public calendar), with a name, category and privacy. Imported events are read-only in Klander and show where they came from.
- **import-ics Edge Function:** downloads and parses the link (see 2.4), upserts the events and removes stale ones. You can run it from **Sync now**, and it also runs automatically every 3 hours via `pg_cron`.
- **feed Edge Function:** publishes your Klander-made events as an .ics file at a secret link. Buttons subscribe to it from Apple Calendar (`webcal://`), Google Calendar or Outlook. It writes a London time-zone definition so repeating events stay correct across clock changes in other apps.



Linked calendars



An imported lecture



Activity feed

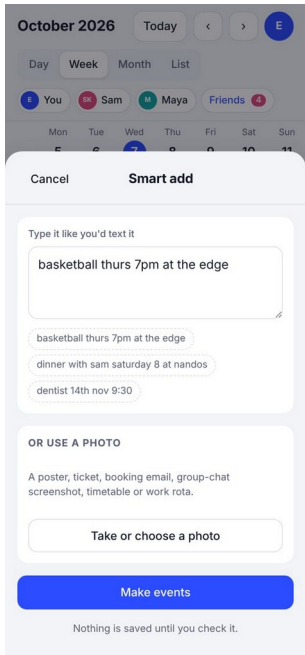
Problems and fixes

Problem	Fix
A database change that dropped an index needed manual approval and timed out	Rewrote it as a change that only adds things (a new unique constraint), which applied straight away
The first deployment of import-ics accidentally contained a placeholder instead of the shared parser file	Spotted straight away and redeployed with the real file before anything used it
Outlook calendars use Windows time-zone names that the parser doesn't know	Falls back to the user's own zone (Europe/London); tested with "GMT Standard Time"
A calendar link could point at a private address	The function refuses localhost and private network ranges

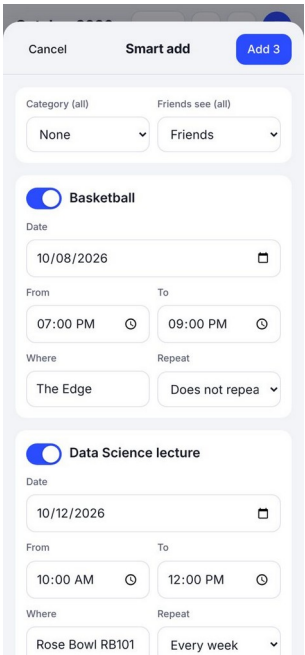
3.6 Phase 5: Smart add

What was built

- **Smart add sheet:** type a sentence (with example chips to tap), take or choose a photo, or paste a screenshot.
- **smart-add Edge Function:** calls Gemini with a strict JSON schema and UK-specific rules, falls back to another model if one is busy, validates the reply and limits each person to 30 requests a day.
- **Review screen:** each event becomes an editable card with a switch to leave it out. You can set category and privacy for all at once, and anything the AI is less than 60% sure about is flagged. Timetables become weekly repeats on the right days.



Typing a sentence



Review before saving



Saved to the calendar

My role

- Chose Gemini's free tier after comparing costs, created the API key in Google AI Studio and added it as the `GEMINI_API_KEY` secret in Supabase.
- Tested it live with real sentences and screenshots.

Problems and fixes

Problem	Fix
On a phone, the date and time boxes on the review cards were wider than the screen	Put the date on its own row and the times in a two-column row; inputs can now shrink
After pushing, Smart add didn't appear on my phone	The installed app was still showing its cached version; it updated after a short wait and a refresh
My Gemini key didn't start with "Alza" as expected	Newer Google keys use a different format; it worked as soon as it was saved

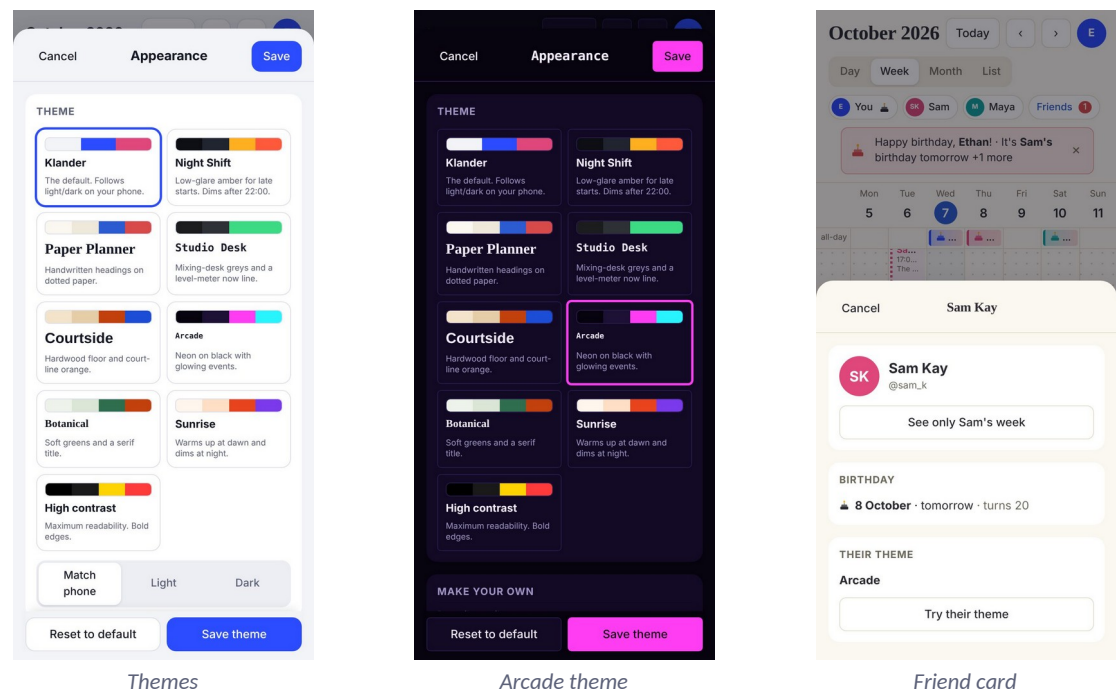
3.7 Version control and deployment

Commit	Contents
7c2897c	Klander v0.1: accounts, calendar views, events, recurring, categories, PWA
5d3bce3	Add friends: requests, invite links, friends' week overlay, privacy
4a37ce7	Calendar links import, Klander subscribe feed, live updates, friends activity feed
56bb07c	Smart add: type or photo to events with Gemini

Each `git push` triggers a new Vercel build. Database changes were applied as named migrations, and the full schema is in Appendix A.

3.8 Phase 6: Themes and birthdays

- **Nine themes** (Klander, Night Shift, Paper Planner, Studio Desk, Courtside, Arcade, Botanical, Sunrise, and High contrast), a custom theme builder, a theme made from a photo (colours picked on the phone with k-means), and an AI "describe a vibe" theme.
- **A contrast guard:** whatever colours a theme uses, text is pushed until it reaches a readable contrast ratio, so a friend's yellow-on-white theme can't make the app unreadable. Themes can be shared as a code and tried from a friend's card.
- **Birthdays:** add yours once and friends see it on the calendar every year, with the age shown only if you allow it. Leap-day birthdays fall on 28 February in other years. On the day: a banner, a cake on your avatar and optional confetti.
- **UK dates:** bank holidays, Easter and Bonfire Night are labelled automatically.

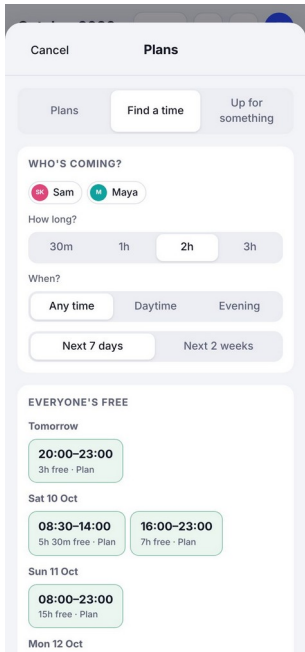


Problem	Fix
The Sunrise theme had low-contrast text at night	Tested every theme in light and dark at four times of day (200 checks) and strengthened the night colours
A shared theme code could carry junk or CSS	Codes are cleaned: only known keys, valid hex colours, and a size limit

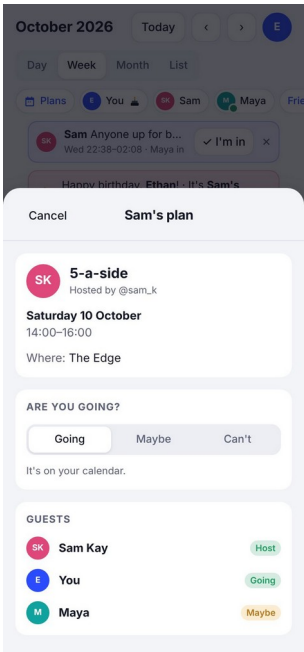
3.9 Phase 7: Friends and plans

- **Plans → Find a time:** pick friends, how long and when; Klander suggests times when everyone (or everyone but one) is free.
- **Invites and RSVPs:** Going / Maybe / Can't, a guest list on the event, and plans you're going to appear on your calendar.
- **"Up for something?"** posts that friends answer with "I'm in", plus a green dot for friends who are free right now and an optional status.

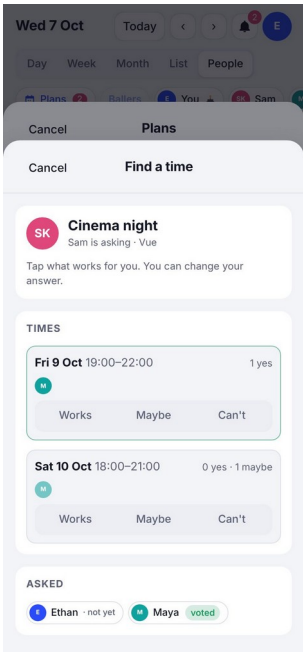
- **Part 2:** a People view (everyone side by side for one day), friend groups, Close friends, surprise plans, time polls with votes, countdowns, and "catch up" nudges for friends I haven't made plans with in 3 weeks.



Find a time



An invite



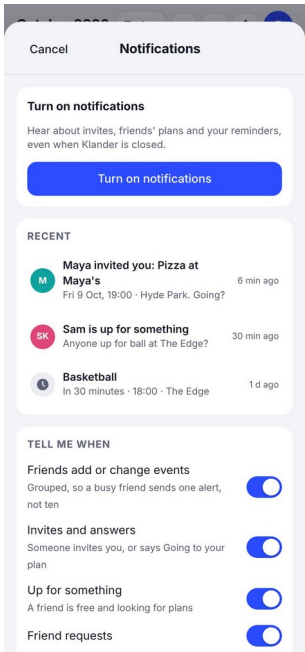
A time poll

Problem	Fix
Saving a poll failed with a permissions error	Inserting and reading back the new row needs the owner to be allowed to read it straight away; added the owner to the read rule
The app crashed if the busy-times function returned nothing	Checked the result is a list before using it

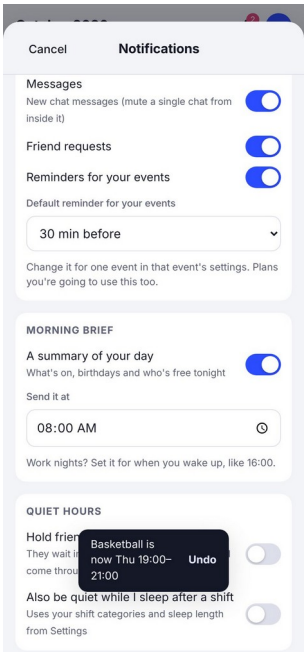
3.10 Phase 10 part 1: Notifications (brought forward)

I asked whether notifications worked yet. They didn't, and without them friends had to keep opening the app, so I moved this part forward.

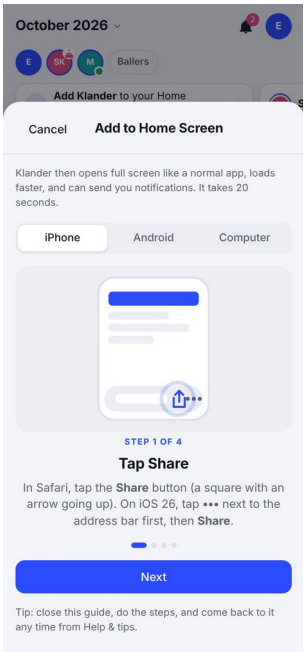
- **Lock-screen notifications** using Web Push. On iPhone this only works once Klander is on the Home Screen.
- **What notifies you:** friends adding events (grouped, so five quick changes make one alert), invites and answers, "up for something", friend requests, poll votes, messages, and reminders.
- **The bell:** an inbox of the last 30 days, a setting for each type, quiet hours, muting a friend, and a test button.



Notification settings



Morning brief settings



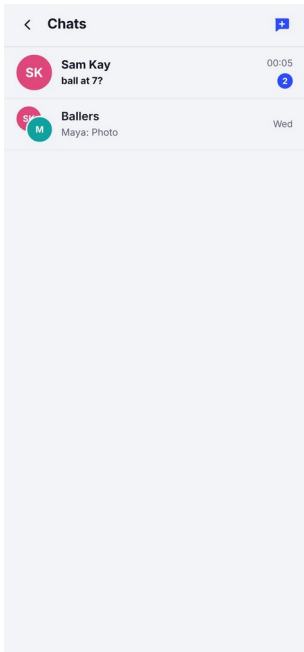
Home Screen guide

Problem	Fix
The push function failed to start (BOOT_ERROR)	The repeat-rule library had to be imported differently on the server
Reminders were never being created (found later, in phase 10 part 2)	The "only once" rule used a partial index the database couldn't use for upserts. Replaced it with a unique dedupe_key, then tested it live on my own phone

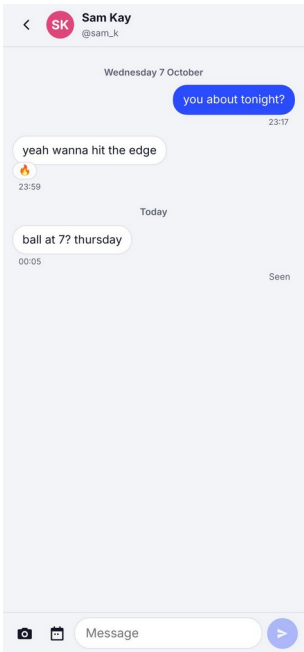
3.11 UI v2 and phase 8: Chat

I said the buttons were too close together and I wanted to swipe between weeks, so the redesign in 2.8 shipped with chat.

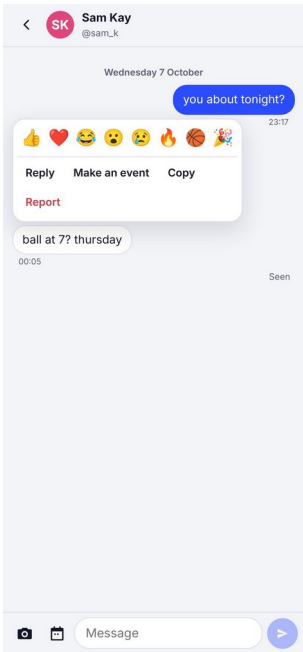
- **Chats:** direct messages, groups and one chat per plan. Messages can have replies, reactions, photos and event cards, and you can see when they've been read.
- **Make an event from a message:** tap "ball at 7 thursday" and Smart add fills in the event.
- **Safety:** mute, leave, block and report.
- **Storage cleanup:** I asked where chat photos are kept. They're now deleted after 6 months, or straight away if the message is deleted, by a nightly cleanup job.



Chats



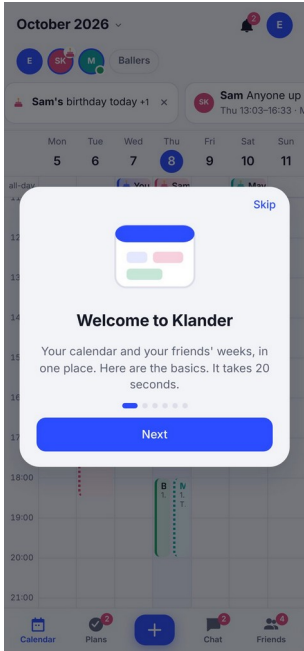
A conversation



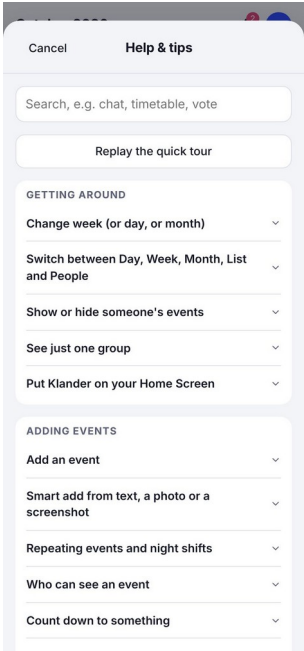
Message actions

3.12 Help and welcome tour

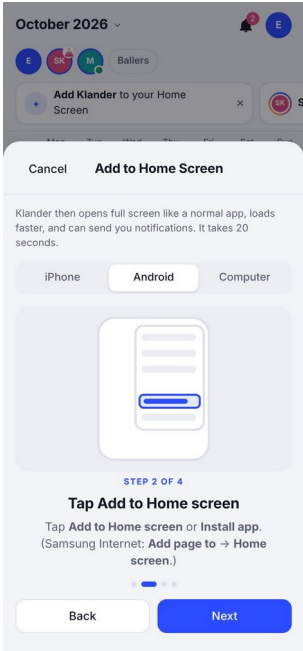
I suggested a tutorial for new users and people who forget features. There is now a six-slide welcome tour and a searchable **Help & tips** page. Each tip has a "Show me" button that opens the right screen.



Welcome tour



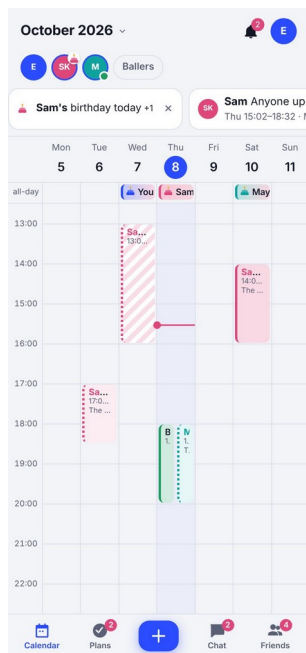
Help & tips



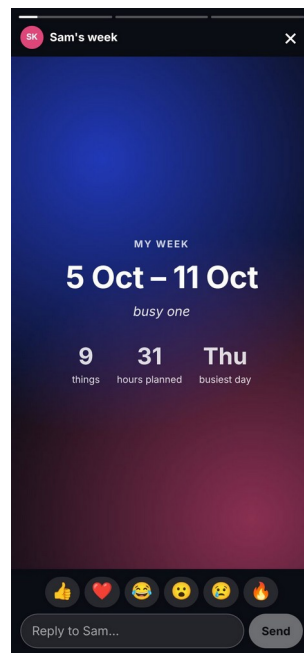
Install steps (Android)

3.13 Phase 9: Memories and My Week

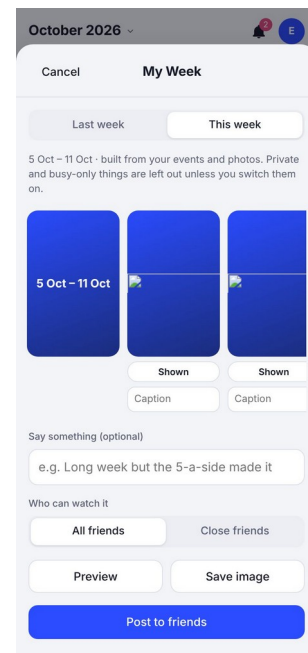
- **Photos on events:** anyone going to a plan can add theirs. Photos are shrunk on the phone before upload.
- **My Week:** a story built from the week's photos and plans, with a cover, day slides, "the week in numbers" and who you saw. You can hide or caption anything, choose All friends or Close friends, and save it as an image to post anywhere.
- **Friends' weeks** appear as cards on the home screen. You can react or reply, which goes to chat. There's also a "One year ago" card.



Friends' weeks on home



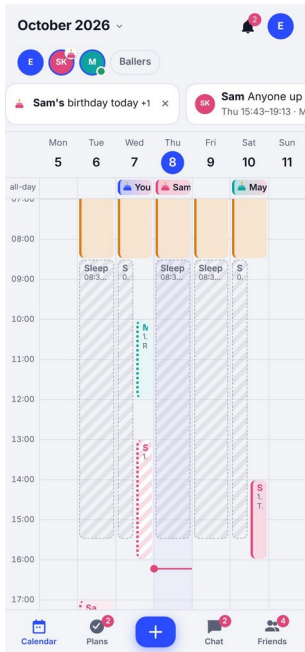
A My Week story



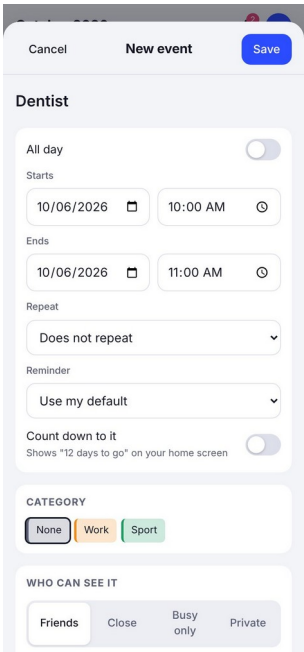
Building My Week

3.14 Phase 10 part 2: Shifts, family link and smarter editing

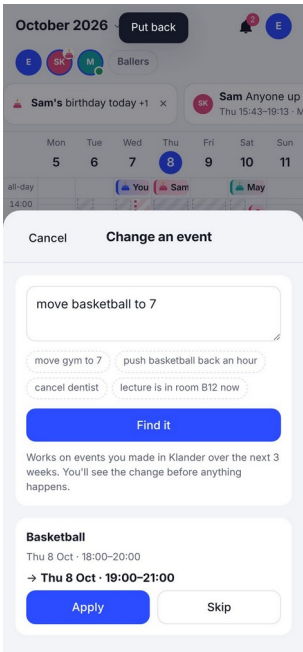
- **Shift categories:** marking Work as shifts adds a Sleep block after each one. The free-time finder treats my sleep, and my friends' sleep, as busy, and notifications can wait until I'm awake.
- **Family link:** a calendar link for my mum, with Busy-only and Close events shown as "Busy" and Private events never included.
- **Morning brief:** an optional daily notification with my day, birthdays and who's free tonight.
- **Clash warnings** before saving, **Undo** after deleting, **category reminders**, and **Change an event** by typing, for example "move gym to 7", which shows the change before applying it.
- **Two-way Apple Calendar sync was skipped.** It would have needed my Apple app-specific password, which can read all my iCloud calendars. I decided the risk wasn't worth it when the family link already covers my mum.



Sleep after shifts



Clash warning



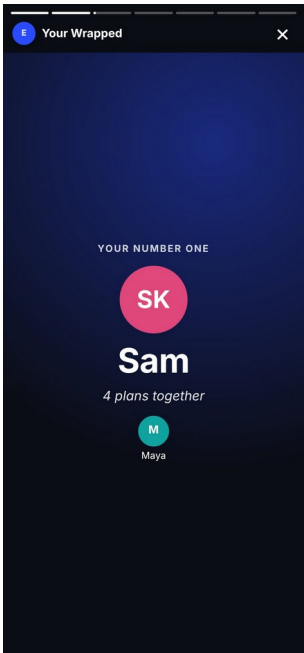
Change by typing

3.15 Phase 11: Insights

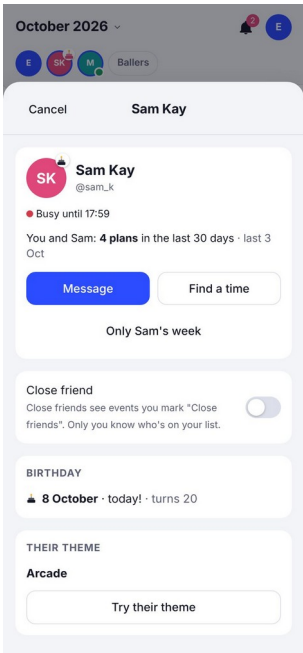
- **Insights** (private to me): hours by category, busiest day and week, for this month, term or year.
- **Streaks**, e.g. "Sport, 6 weeks running".
- **Who I made plans with**, with a count on each friend's card that only the two of us can see.
- **Klander Wrapped**: an end-of-term story with my top category, my number one friend, a favourite photo, my longest streak and my busiest week, which I can save as an image.



Insights



Wrapped story

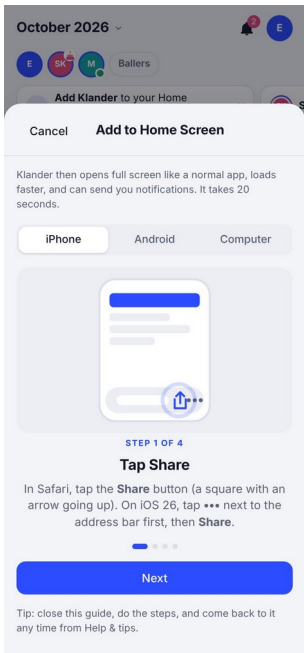


Hangouts on a friend card

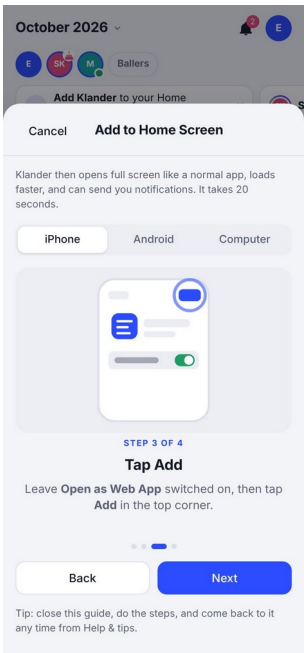
Problem	Fix
The period buttons were squashed flat inside the scrolling sheet	Stopped that row being its own scroll area
A new card at the front of the card row left the row scrolled to the middle	Reset the row to the start when cards change, and turned off scroll anchoring

3.16 Add to Home Screen guide

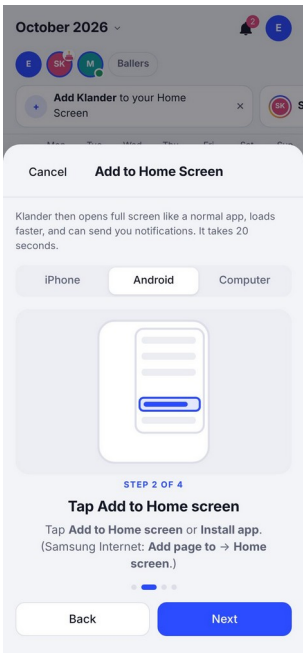
Most of my friends use iPhones, and notifications only work once Klander is on the Home Screen, so I asked for a tutorial. It works out the phone type and shows drawn, step-by-step pictures for iPhone Safari (including the iOS 26 ●●● menu), iPhone Chrome, Android (with a one-tap Install button when the phone supports it) and computers. It opens after the welcome tour, from a dismissible home card, from the bell, from Settings and from Help.



Step 1: Share



Step 3: Add



Android

3.17 Version control (phases 6–11)

Commit	Contents
3fe8f0c	Project report, How Klander works guide, consolidated schema
dc03d88	Phase 6: themes and birthdays
f5d2404	Phase 7: free-time finder, plans with RSVPs, up for something
2035c50	Notifications: push alerts, inbox, reminders, quiet hours
501322c	Phase 7 part 2: side by side, groups, close friends, polls, countdowns, catch-ups
ee2a1fe	Cleaner layout with swipe, and phase 8: chat
76d4d3b	Chat photo cleanup

Commit	Contents
f2b4d0f	Welcome tour and Help & tips
48e39e6	Phase 9: memories and My Week
932b344	Phase 10 part 2: family link, shifts and sleep, morning brief, clashes, undo, edit by sentence
60e2551	Phase 11: Insights, streaks, hangout stats and Klander Wrapped
7b34b90	Add to Home Screen guide with pictures

4. Testing

Testing happened at four levels: database security, logic (unit tests), live server functions, and the user interface. Section 4.6 covers my own real-world testing. Every result below is the actual output recorded when the test was run.

4.1 Database security tests

These tests run inside the database while pretending to be a particular signed-in user, or a logged-out visitor. Test users are created inside the test and everything is rolled back at the end, so the live database is untouched.

Phase 2: your own data

#	Test	Expected	Actual	Result
S1	User inserts their own repeating event	Allowed	1 event saved	Pass
S2	New user gets default categories on sign-up	5 categories	5 categories	Pass
S3	A different user tries to read the first user's events	0 rows	0 rows	Pass
S4	A different user tries to read their categories	0 rows	0 rows	Pass
S5	A different user tries to rename their profile	0 rows changed	0 rows changed	Pass
S6	A different user tries to delete their events	0 rows deleted	0 rows deleted	Pass
S7	Logged-out visitor reads events / profiles	0 / 0	0 / 0	Pass
S8	Supabase security advisor	No warnings	No warnings	Pass

Phase 4: friends and privacy (Alice, Bob, Carl)

Bob has three events: Gym (Friends, location "The Edge", with a private note), Doctor (Busy only) and Diary (Private).

#	Test	Expected	Actual	Result
F1	Alice views Bob's events before they are friends	0	0	Pass
F2	Alice sends a request to bob_t	sent	sent	Pass
F3	Alice sends the same request again	already_sent	already_sent	Pass
F4	Alice requests a username that doesn't exist	not_found	not_found	Pass
F5	Alice requests herself	self	self	Pass
F6	Alice tries to accept her own request	0 rows	0 rows	Pass
F7	Alice tries to insert an accepted friendship directly	Blocked	Blocked	Pass
F8	Alice views Bob's events while the request is pending	0	0	Pass
F9	Bob accepts	1 row	1 row	Pass

#	Test	Expected	Actual	Result
F10	Alice now views Bob's events	Busy (no location), Gym/The Edge; no Diary	Busy/-, Gym/The Edge	Pass
F11	Alice reads Bob's events table directly	0	0	Pass
F12	Alice reads anyone's invite codes	0	0	Pass
F13	Carl uses a fake invite code	invalid	invalid	Pass
F14	Carl uses Bob's real invite code	friends with bob_t; sees 2 events	bob_t; 2 events	Pass
F15	Carl sees only his own friendships	1	1	Pass
F16	Logged-out visitor calls friend_events	Blocked	Blocked	Pass

Activity feed and calendar links

#	Test	Expected	Actual	Result
A1	Bob adds Gym (Friends), Doctor (Busy), Diary (Private), then makes Gym private and Diary public	added Gym, added (no title), removed, added Diary	added:Gym, added:-, removed:-, added:Diary	Pass
A2	Bob tries to write a fake activity row as Alice	Blocked	Blocked	Pass
A3	Bob adds a calendar link to Alice's account	Blocked	Blocked	Pass
A4	Alice reads others' calendar links / feed tokens	0 / 0	0 / 0	Pass
A5	A stranger reads activity	0	0	Pass
A6	A user calls the cron-secret check or reads private secrets	Blocked	Blocked	Pass

4.2 Unit tests: logic

Repeating events across the clock change

Weekday night shift 23:30–08:30 from Monday 19 October 2026, repeating until 30 October, with Wednesday 21 October skipped. The clocks go back on Sunday 25 October.

#	Test	Expected	Actual	Result
R1	Occurrences before the clock change	23:30 → 08:30	Mon 19, Tue 20, Thu 22, Fri 23 at 23:30	Pass
R2	Occurrences after the clock change	Still 23:30	Mon 26 – Fri 30 at 23:30	Pass
R3	Skipped date	Wed 21 missing	Wed 21 missing	Pass
R4	No weekend occurrences	Sat/Sun absent	Absent	Pass
R5	Repeat rule round-trip	weekdays until 2026-10-30	preset weekdays, until 2026-10-30	Pass

#	Test	Expected	Actual	Result
R6	Overlap layout: shift tail (00:00–08:30) and 07:00–09:00 event	Side by side, 2 columns	col 0 and col 1 of 2	Pass

Calendar file (.ics) parser

#	Input	Expected	Actual	Result
I1	Weekly lecture 10:00 Europe/London in October	09:00 UTC (BST)	09:00 UTC	Pass
I2	EXDATE 26 Oct 10:00 (after clock change)	10:00 UTC skipped	10:00 UTC in exdates	Pass
I3	Moved occurrence (RECURRENCE-ID)	Own event + original skipped	Separate row + exdate added	Pass
I4	All-day reading week 2–6 Nov	All-day, ends 7 Nov	all_day true, ends 7 Nov	Pass
I5	Outlook "GMT Standard Time" 15:00	15:00 London	15:00 UTC (GMT)	Pass
I6	UTC time 09:00Z	09:00 UTC	09:00 UTC	Pass
I7	Event from 2020	Ignored (too old)	Not imported	Pass
I8	STATUS:CANCELLED	Ignored	Not imported	Pass
I9	Escaped comma in title	"Intro to Data Science, Lecture"	Correct	Pass
I10	Klander feed → parse again (round trip)	Same times, rule and skip	Identical	Pass
I11	Long title in feed	Folded at 75 bytes	Folded correctly	Pass

Smart add: AI draft to event

#	Draft from AI	Expected event	Actual	Result
M1	Basketball 8 Oct 19:00, no end	19:00–20:00 (1 hour default)	18:00Z–19:00Z (BST)	Pass
M2	Night shift 23:30–08:30, weekdays	Ends next day; Mon–Fri rule	Ends 13 Oct; BYDAY=MO..FR	Pass
M3	Lecture weekly Mon & Thu until 11 Dec	Two days a week	Mon 12, Thu 15, Mon 19, Thu 22...	Pass
M4	Reading week 2–6 Nov, all day	All-day to 7 Nov	Correct	Pass

4.3 Live server tests

These ran against the real deployed functions, triggered from inside the database with pg_net, using a test account.

#	Test	Expected	Actual	Result
L1	Scheduled sync of the gov.uk UK bank holidays calendar	Imports holidays with correct dates	200 OK: 12 events, e.g. Summer bank holiday 31 Aug (23:00Z on the 30th)	Pass
L2	Subscribe feed with a valid token	Valid .ics with London time zone; only Klander-made events	200 OK; night shift at 23:30 London; no bank holidays	Pass
L3	Subscribe feed with a wrong token	Not found	404 Not found	Pass
L4	Smart add with real sentences after adding the key	Correct drafts	Worked (my own testing)	Pass

4.4 Automated interface tests

A headless Chromium browser at iPhone size (390 × 844, UK time zone) clicked through the app against a simulated backend and saved a screenshot of each step. The screenshots in Section 3 come from these runs.

#	Journey	Expected	Result
U1	Sign in → first-time setup → calendar	Reaches the week view	Pass
U2	Create weekday night shift and a weekly event	Both appear, overnight across days	Pass
U3	Switch Day / Week / Month / List	Same events in every view	Pass
U4	Delete one repeat	Asks "only this one or every repeat"	Pass
U5	Dark mode	All text readable	Pass
U6	Open app from an invite link	Invite notice on sign-in; "You're now friends" after	Pass
U7	Friends' events, busy block, hide a friend	Coloured, striped, hidden when toggled	Pass
U8	Add calendar link → import → open imported event	Count shown; event is read-only	Pass
U9	Get subscribe link	Apple, Google and Outlook buttons	Pass
U10	Smart add sentence → review → save	3 drafts, low-confidence one flagged, saved	Pass
U11	No sideways scrolling at 390 px	Page width 390	Pass (after layout fix)
U12	No JavaScript errors in any journey	None	Pass

4.5 Bugs found by testing

Found by	Bug	Status
UI test	Week view event text centred vertically	Fixed
UI test	Visibility buttons wrapping	Fixed

Found by	Bug	Status
UI test	Overnight shift label confusing on its second day	Fixed
UI test	Smart add review cards wider than the phone	Fixed
Unit test	Repeat end date shifted by time-zone conversion	Fixed
Me	Google sign-in redirect_uri_mismatch	Fixed (configuration)
Me	Smart add not showing after deploy	Cache; resolved itself

4.6 My real-world testing

- Ran the app locally, then live on klander.vercel.app on my Mac and iPhone.
- Created my real account, signed in with Google, and added my night shift and other events.
- Created a second account in a private window to act as a friend, checked that live updates, busy blocks and private events behaved correctly, and confirmed that "everything seems to be working fine" before inviting anyone.
- Tested Smart add with real sentences once the Gemini key was set.

Limitation: the automated tests couldn't connect to the live Supabase API from the build environment, so UI tests used a simulated backend. To cover the gap, security was tested inside the real database, the server functions were tested live through pg_net, and I tested the real app end to end myself.

4.7 Database privacy tests: phases 6–11

Run against the live database on 8 October 2026 with four test users, all rolled back afterwards. A and B are friends, and B has A on their close friends list; C is friends with both; D is a stranger. B has a Close event ("Close plan" at "Secret spot"), a surprise hidden from A, and a normal plan ("Pizza").

#	Test	Expected	Actual	Result
P1	A reads friends' birthdays (B hid their year)	14 March, no year	3/14/noyear	Pass
P2	Stranger D reads friends' birthdays	0	0	Pass
P3	A (on B's close list) views B's events	Close plan with place; Pizza; no surprise	Close plan@Secret spot, Pizza	Pass
P4	C (not close) views B's events	Busy (no place); surprise; Pizza	Busy@-, Surprise for A, Pizza	Pass
P5	B invites stranger D	Blocked	Blocked	Pass
P6	B invites A to the event hidden from A	Blocked	Blocked	Pass
P7–P9	B invites A; A sees the invite and answers Going	Allowed	ok; 1; 1 row	Pass
P10	C reads the guest list of a plan they're not in	0	0	Pass
P11	C changes A's answer	0 rows	0 rows	Pass

#	Test	Expected	Actual	Result
P12–13	"Up for something": friend A / stranger D	1 / 0	1 / 0	Pass
P14	A starts a DM with stranger D	Blocked	Blocked	Pass
P15–16	B / C read A and B's DM	1 / 0	1 / 0	Pass
P17	C posts into A and B's DM	Blocked	Blocked	Pass
P18	C sends a message pretending to be A	Blocked	Blocked	Pass
P19	A messages B after B blocks A	Blocked	Blocked	Pass
P20–22	Photo on A's event: friend B / stranger D / D reads photo rows	true / false / 0	true / false / 0	Pass
P23–25	A posts a Close-friends week: B / C / D	1 / 0 / 0	1 / 0 / 0	Pass
P26–28	D reads others' family links / notifications / close lists	0 / 0 / 0	0 / 0 / 0	Pass
P29	Notifications created for A by B's actions	friend events, invite, broadcast, accepted	4: accepted, broadcast, friend_event, invite	Pass
H1–H5	hangout_counts: A, B, a guest who isn't a friend, after a block, signed out	1 / 1 / none / none / none	2=1, 1=1, none, none, none	Pass

4.8 Logic tests: phases 6–11

Automated tests of the logic, run with the UK time zone. These are counts of individual checks.

Area	What it checks	Checks	Result
Themes	Every theme in light and dark at 03:00, 07:00, 13:00 and 20:00 meets contrast targets; bad custom themes get fixed; theme codes round-trip and reject junk	200	Pass
Birthdays and UK dates	Leap-day birthdays, birthdays across New Year, hidden ages, "in N days" across the clock change; Easter, bank holidays, Bonfire Night	17	Pass
Free-time finder	Merging, gaps, "free until", night shift still 23:30 after the clocks change, all-day events ignored, everyone / all-but-one slots, rounding	16	Pass
Plans	Catch-up nudges and poll vote tallies	3	Pass
My Week	Private titles never appear, busy titles never appear, private photos hidden, who-I-saw, edits kept	10	Pass
Shifts and editing	Sleep blocks (including after the clock change), friends' sleep, "move to 7" keeps the length, overnight ends roll over	10	Pass
Insights	Streak counting (including Mondays and broken streaks), shifts excluded, future events excluded, Wrapped slides	13	Pass

Total: 269 checks, 0 failures.

4.9 Interface tests: phases 6–11

#	Journey (390 × 844, simulated backend)	Result
U13	Switch themes, try a friend's theme, then undo	Pass
U14	Find a time → pick a slot → invite → save	Pass
U15	Post "up for something"; poll with votes; pick the winning time	Pass
U16	Notifications inbox, settings, open a notification	Pass
U17	Swipe weeks; title menu; bottom tabs	Pass
U18	Chat: open, react, reply, make an event from a message, share a photo	Pass
U19	Welcome tour, Help search, "Show me"	Pass
U20	My Week: build, hide, preview, post; watch a friend's week	Pass
U21	Sleep blocks shown; clash warning; delete then Undo; change by typing; family link; morning brief	Pass
U22	Insights for month and term; play Wrapped; hangout line on a friend card	Pass
U23	As an iPhone and as an Android phone: tour → install guide, all steps, dismiss card, bell button	Pass
U24	No JavaScript errors in any journey	Pass

4.10 Bugs found in phases 6–11

Found by	Bug	Status
Logic test	Sunrise theme too low-contrast at night	Fixed
Database test	New polls and photos couldn't be read back after saving	Fixed
Database test	Chat test hung because it needed approval to delete	Test rewritten
Live test	Push function wouldn't start	Fixed
Live test	Reminders silently never created	Fixed (unique dedupe_key)
UI test	Crash when busy times came back empty	Fixed
UI test	Message time shown under every bubble	Fixed (end of group only)
UI test	Insights period buttons squashed; card row opening mid-way	Fixed
Me	Buttons too close together; no swiping	Fixed (UI v2)
Me	git push failed with HTTP 400 (large files)	Fixed (http.postBuffer)

Live checks on my real account: my phone received a test morning brief from the push function, and the family link returned my week (74 events) with private events removed. Afterwards I switched the brief off and replaced the test link.

5. Evaluation

5.1 Against the objectives

#	Objective	Evidence	Outcome
1	Own accounts (email / Google), username, colour	U1; my live sign-up and Google sign-in; unique citext usernames	Met
2	Day, week, month, list; create / edit / delete	U2-U3; screenshots in 3.3	Met
3	Repeats incl. overnight, correct after clock change	R1-R5; M2	Met
4	Delete one occurrence or the series	R3; U4	Met
5	Categories and per-event visibility	Editor, settings, S1-S8, F10	Met
6	Free iPhone install, offline viewing	PWA installs from Safari; cached data shows offline	Met (read-only offline)
7	Friends by username or invite link; overlay	F2-F15; U6-U7; tested with a second account	Met
8	Privacy enforced by the database	F1, F8, F10-F12, F16, A1-A6	Met
9	Live updates and activity feed	A1; Realtime tested between two accounts	Met
10	Import links that refresh; publish a subscribe link	I1-I11; L1-L3; cron every 3 h	Met
11	Text / photo / screenshot to reviewable events	M1-M4; U10; my live testing	Met
12	£0 per month	All free tiers (see 5.3)	Met

5.2 What went well

- **Privacy is enforced in the database.** Even with direct API access, nobody can see more than they're allowed to, and I have test evidence for every rule.
- **It fits into the calendars people already use.** It reads uni, Google, Outlook and iCloud links and publishes its own, so my mum can see my plans in Apple Calendar without installing anything.
- **Night shifts and the clock change work correctly.** These are easy to get wrong, and both are covered by tests.
- **Smart add is genuinely faster than typing,** and the review step means a wrong guess never ends up saved.
- **It cost nothing to build or run.**

5.3 Limitations

Limitation	Impact	Plan
Offline is read-only	Can't add events with no signal	Queue changes offline and sync later
Calendar links are one-way in each direction	Editing an Apple event in Klander isn't possible	Two-way sync skipped on purpose (see 3.14)
Gemini free tier	Data may be used to improve Google's models	Switch to the paid tier (~35p/month) before strangers use it
Supabase free projects pause after a week idle	App unavailable until restored	Daily use keeps it awake; Pro plan if it grows
UI tests used a simulated backend	Less end-to-end coverage	Add a test Supabase project for full end-to-end tests
No two-way Apple sync	Edits in Apple Calendar don't come back to Klander	Skipped on purpose (password risk); family link covers the main need
iPhone notifications need the Home Screen app	Friends who only use Safari get no alerts	Step-by-step install guide (3.16)
Chat and live updates not yet tested with my friends at scale	Unknown behaviour with many messages	Test with the group before wider launch
No video export of My Week	Image only	Possible later with MediaRecorder

5.4 Cost and environmental impact

Running costs are £0 a month: Supabase free tier, Vercel Hobby, and the Gemini free tier with a limit of 30 requests per person per day.

I estimated the carbon footprint (all figures approximate): roughly **0.2–1.2 kg CO₂e** to build it, mostly the AI assistance and my own devices, and **1.5–5 kg CO₂e a year** to run it for my friend group. That's similar to driving a petrol car 10–30 km a year. Photos (phase 9) are the biggest increase, so they are compressed on the phone and chat photos are deleted after 6 months.

5.5 What I learned

- How a real full-stack app fits together: front end, authentication, a relational database, serverless functions, scheduling and deployment.
- Why security belongs in the database (Row Level Security) rather than in the app, and how to test it by pretending to be different users.
- How messy real-world data is: time zones, clock changes, Outlook's time-zone names, cancelled and moved events in calendar files.
- How to configure OAuth sign-in, environment variables and API secrets safely.
- How to direct an AI coding assistant effectively: describe the goal clearly, make the decisions myself, check its work and test thoroughly.

5.6 Objectives for phases 6–11

#	Objective	Evidence	Outcome
13	Themes and birthdays	3.8; theme and birthday logic tests; P1–P2	Met
14	Find a time without sharing details	friend_busy_v2 returns times only; free-time tests; U14	Met
15	Plans with invites and answers	P5–P11; U14–U15	Met
16	Close friends and surprises	P3–P4, P6	Met
17	Chat	P14–P19; U18	Met
18	Photos and My Week	P20–P25; My Week tests; U20	Met
19	Lock-screen notifications	Received on my iPhone; P29; U16	Met (iPhone needs Home Screen)
20	Night-shift sleep	Shift tests; U21	Met
21	Family link	Live check: 74 events, no private ones; P26	Met
22	Insights and Wrapped	Insights tests; H1–H5; U22	Met
23	Easy for new people	Tour, Help, install guide; U19, U23	Met
24	£0 a month	All still on free tiers	Met

5.7 Reflection on phases 6–11

- **Testing with my own data found the worst bug.** Reminders had passed every mock test but were never being created on the real server. Only a live test on my own phone showed it.
- **Saying no was a real decision.** Two-way Apple sync was on the plan, but it needed a password that could read all my iCloud calendars. I chose the family link instead.
- **Listening to how it felt to use** led to the biggest improvement: the swipeable redesign came from my own complaint that the buttons were cramped.
- **Privacy got harder with every feature.** Close friends, surprises, chat, photos and stats all needed their own database rules. Writing a test for each rule is what kept it trustworthy.

6. Future development

Phases 6–11 are built (Sections 3.8–3.17). What is left is in my interactive roadmap:

Phase	Main features
12. Launch & portfolio (next)	Privacy page, "download my data" and "delete my account" (UK GDPR), landing page, demo mode with fake friends, accessibility and speed pass, paid AI tier, portfolio case study
Deferred	Travel time ("leave by 18:20"), Training Hub link, My Week video, offline editing
Skipped	Two-way Apple Calendar sync (needs an Apple password that can read every iCloud calendar)

Appendix A: Database schema

The live schema, consolidated from the migrations applied to the [klander](#) Supabase project. It is also saved in the repository as [supabase/schema.sql](#).

```
-- Klander database schema (Supabase / Postgres 17)
-- Consolidated from the migrations applied to project `klander` (davompbqkwiiecfuthx),
-- October 2026.
-- Every table has Row Level Security (RLS) switched on.

create extension if not exists citext with schema extensions;
create extension if not exists pg_net;
create extension if not exists pg_cron;

-----
-- PROFILES: one row per user, created automatically on sign-up
-----
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  username extensions.citext unique check (username ~ '^[a-zA-Z0-9_]{3,20}$'),
  display_name text check (char_length(display_name) <= 50),
  avatar_url text,
  colour text not null default '#2b4cff' check (colour ~ '^[0-9a-fA-F]{6}$'),
  timezone text not null default 'Europe/London',
  theme text not null default 'system' check (theme in ('system','light','dark')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
alter table public.profiles enable row level security;
create policy "Signed-in users can view profiles" on public.profiles for select to
authenticated using (true);
create policy "Users update own profile" on public.profiles for update to authenticated
  using ((select auth.uid()) = id) with check ((select auth.uid()) = id);

-----
-- CATEGORIES: Work, Uni, Sport, Social, Personal (editable)
-----
create table public.categories (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.profiles(id) on delete cascade,
  name text not null check (char_length(name) between 1 and 30),
  colour text not null check (colour ~ '^[0-9a-fA-F]{6}$'),
  sort int not null default 0,
  created_at timestamptz not null default now()
);
alter table public.categories enable row level security;
create policy "Users manage own categories" on public.categories for all to authenticated
  using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

-----
-- CALENDAR SOURCES: linked .ics / webcal calendars (uni timetable etc.)
-----
create table public.calendar_sources (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.profiles(id) on delete cascade,
  name text not null check (char_length(name) between 1 and 60),
```

```

url text not null check (url ~* '^(https?|webcal)://'),
category_id uuid references public.categories(id) on delete set null,
visibility text not null default 'busy' check (visibility in
('friends','busy','private')),
last_synced_at timestamptz,
last_error text,
event_count int not null default 0,
created_at timestamptz not null default now()
);
alter table public.calendar_sources enable row level security;
create policy "Users manage own calendar sources" on public.calendar_sources for all to
authenticated
    using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

-----
-- EVENTS
-----
create table public.events (
    id uuid primary key default gen_random_uuid(),
    owner_id uuid not null references public.profiles(id) on delete cascade,
    title text not null check (char_length(title) between 1 and 300),
    notes text check (char_length(notes) <= 2000),
    location text check (char_length(location) <= 200),
    starts_at timestamptz not null,
    ends_at timestamptz not null,
    all_day boolean not null default false,
    category_id uuid references public.categories(id) on delete set null,
    visibility text not null default 'friends' check (visibility in
('friends','busy','private')),
    rrule text,                                -- repeat rule, e.g.
FREQ=WEEKLY;BYDAY=MO,TU,WE,TH,FR
    exdates timestamptz[] not null default '{}', -- skipped occurrences
    source text not null default 'manual' check (source in
('manual','text','photo','import')),
    source_id uuid references public.calendar_sources(id) on delete cascade,
    external_uid text,                        -- UID from the imported .ics
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    check (ends_at >= starts_at),
    constraint events_source_uid_key unique (source_id, external_uid)
);
create index events_owner_start_idx on public.events(owner_id, starts_at);
alter table public.events enable row level security;
create policy "Users manage own events" on public.events for all to authenticated
    using ((select auth.uid()) = owner_id) with check ((select auth.uid()) = owner_id);

-----
-- FRIENDS
-----
create table public.friendships (
    id uuid primary key default gen_random_uuid(),
    requester uuid not null references public.profiles(id) on delete cascade,
    addressee uuid not null references public.profiles(id) on delete cascade,
    status text not null default 'pending' check (status in ('pending','accepted')),
    created_at timestamptz not null default now(),
    check (requester <> addressee)
);

```

```

create unique index friendships_pair_idx on public.friendships (least(requester,
addressee), greatest(requester, addressee));
alter table public.friendships enable row level security;
create policy "See own friendships" on public.friendships for select to authenticated using
((select auth.uid()) in (requester, addressee));
create policy "Accept requests sent to you" on public.friendships for update to
authenticated
    using ((select auth.uid()) = addressee) with check ((select auth.uid()) = addressee and
status = 'accepted');
create policy "Remove own friendships" on public.friendships for delete to authenticated
using ((select auth.uid()) in (requester, addressee));
revoke insert, update on public.friendships from authenticated, anon;
grant update (status) on public.friendships to authenticated;  -- only the status column

create table public.invite_codes (
    code text primary key default encode(extensions.gen_random_bytes(9), 'hex'),
    user_id uuid not null unique references public.profiles(id) on delete cascade,
    created_at timestamptz not null default now()
);
alter table public.invite_codes enable row level security;
create policy "See own invite code" on public.invite_codes for select to authenticated
using ((select auth.uid()) = user_id);

create table public.feed_tokens (
    user_id uuid primary key references public.profiles(id) on delete cascade,
    token text not null unique default encode(extensions.gen_random_bytes(18), 'hex'),
    created_at timestamptz not null default now()
);
alter table public.feed_tokens enable row level security;
create policy "See own feed token" on public.feed_tokens for select to authenticated using
((select auth.uid()) = user_id);

create table public.activity (
    id bigint generated always as identity primary key,
    actor uuid not null references public.profiles(id) on delete cascade,
    verb text not null check (verb in ('added','changed','removed','synced')),
    title text,
    starts_at timestamptz,
    created_at timestamptz not null default now()
);
alter table public.activity enable row level security;
create policy "Friends see activity" on public.activity for select to authenticated using
(public.is_friend(actor));

create table public.ai_usage (
    id bigint generated always as identity primary key,
    user_id uuid not null references public.profiles(id) on delete cascade,
    kind text not null check (kind in ('text','photo')),
    ok boolean not null default true,
    created_at timestamptz not null default now()
);
alter table public.ai_usage enable row level security;
create policy "See own AI usage" on public.ai_usage for select to authenticated using
((select auth.uid()) = user_id);

```

```

-----
-- FUNCTIONS (security definer = run with owner rights, each checks auth.uid())
-----

```

```

create function public.is_friend(other uuid) returns boolean language sql stable security
definer set search_path = '' as $$
    select exists (select 1 from public.friendships f where f.status = 'accepted'
        and ((f.requester = (select auth.uid()) and f.addressee = other) or (f.addressee =
(select auth.uid()) and f.requester = other)));
$$;

-- Friends' events with privacy applied: private never returned, busy shows only "Busy",
notes never shared
create function public.friend_events()
returns table (id uuid, owner_id uuid, title text, location text, starts_at timestamptz,
ends_at timestamptz,
                all_day boolean, visibility text, rrule text, exdates timestamptz[])
language sql stable security definer set search_path = '' as $$
    select e.id, e.owner_id,
        case when e.visibility = 'busy' then 'Busy' else e.title end,
        case when e.visibility = 'busy' then null else e.location end,
        e.starts_at, e.ends_at, e.all_day, e.visibility, e.rrule, e.exdates
    from public.events e
    join public.friendships f on f.status = 'accepted'
        and ((f.requester = (select auth.uid()) and f.addressee = e.owner_id)
            or (f.addressee = (select auth.uid()) and f.requester = e.owner_id))
    where e.visibility <> 'private';
$$;

-- Also defined (see repo migrations for full bodies):
-- send_friend_request(username) -> 'sent' | 'accepted' | 'already_sent' |
'already_friends' | 'not_found' | 'self'
-- my_invite_code()             -> creates/returns your invite code
-- accept_invite(code)          -> instantly friends with the code's owner
-- my_feed_token(reset bool)    -> creates/returns (or resets) your subscribe link
token
-- handle_new_user() trigger on auth.users: creates profile + 5 default categories
-- log_event_activity() trigger on events: writes privacy-safe rows to activity (skips
imported events)
-- touch_updated_at() trigger on profiles/events
-- check_cron_secret(s) (service_role only) + private.app_secrets for the scheduled sync

-----
-- STORAGE + REALTIME + SCHEDULE
-----

-- avatars bucket: public read; users can only write inside a folder named after their own
id
-- alter publication supabase_realtime add table public.activity, public.friendships;
-- select cron.schedule('klander-calendar-refresh', '7 */3 * * *', 'select
private.run_calendar_refresh()');

-- =====
-- Phase 6: themes & birthdays
-- =====

-- Theme settings (cosmetic, fine for signed-in users to read so friends can "try your
theme")
alter table public.profiles add column theme_config jsonb not null default '{} '::jsonb;
alter table public.profiles add constraint profiles_theme_config_size check
(pg_column_size(theme_config) < 4000);

-- Birthdays live in their own owner-only table (profiles are readable by every signed-in
user)

```

```

create table public.birthdays (
    user_id uuid primary key references public.profiles(id) on delete cascade,
    birthday date not null check (birthday > date '1900-01-01' and birthday <= current_date),
    show_year boolean not null default false,
    updated_at timestamptz not null default now()
);
alter table public.birthdays enable row level security;
create policy "Users manage own birthday" on public.birthdays for all to authenticated
    using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

-- Friends' birthdays: day + month always, year only if that friend chose to show it
create or replace function public.friend_birthdays()
returns table (user_id uuid, month int, day int, year int)
language sql stable security definer set search_path = '' as $$
    select b.user_id, extract(month from b.birthday)::int, extract(day from b.birthday)::int,
        case when b.show_year then extract(year from b.birthday)::int end
    from public.birthdays b
    where public.is_friend(b.user_id);
$$;
revoke execute on function public.friend_birthdays() from public, anon;
grant execute on function public.friend_birthdays() to authenticated;

-- Smart add gains a "theme" mode
alter table public.ai_usage drop constraint if exists ai_usage_kind_check;
alter table public.ai_usage add constraint ai_usage_kind_check check (kind in
('text', 'photo', 'theme'));

-- =====
-- Phase 7: plans, invites, free time, "up for something"
-- =====
alter table public.events add column hidden_from uuid[] not null default '{}'; -- surprise
plans

-- friend_events() now also skips events hidden from the caller:
-- ... where e.visibility <> 'private' and not ((select auth.uid()) =
any(e.hidden_from));
-- log_event_activity() now leaves the title out for busy-only events AND for surprise
events (hidden_from not empty).

-- Free/busy for the free-time finder: times only, never titles. Private events count as
busy too.
create or replace function public.friend_busy()
returns table (owner_id uuid, starts_at timestamptz, ends_at timestamptz, all_day boolean,
rrule text, exdates timestamptz[])
language sql stable security definer set search_path = '' as $$
    select e.owner_id, e.starts_at, e.ends_at, e.all_day, e.rrule, e.exdates
    from public.events e
    where public.is_friend(e.owner_id)
        and (e.rrule is not null or e.ends_at > now() - interval '1 day');
$$;
revoke execute on function public.friend_busy() from public, anon;
grant execute on function public.friend_busy() to authenticated;

create table public.event_invites (
    event_id uuid not null references public.events(id) on delete cascade,
    user_id uuid not null references public.profiles(id) on delete cascade,
    status text not null default 'invited' check (status in
('invited', 'going', 'maybe', 'declined')),

```

```

    responded_at timestamptz,
    created_at timestamptz not null default now(),
    primary key (event_id, user_id)
);
alter table public.event_invites enable row level security;
create policy "Owner and guest see invite" on public.event_invites for select to
authenticated
    using (user_id = (select auth.uid()))
        or exists (select 1 from public.events e where e.id = event_id and e.owner_id =
(select auth.uid()));
create policy "Owner invites friends" on public.event_invites for insert to authenticated
    with check (status = 'invited' and public.is_friend(user_id)
        and exists (select 1 from public.events e where e.id = event_id and e.owner_id =
(select auth.uid())
            and not (user_id = any(e.hidden_from))));
create policy "Owner removes invite" on public.event_invites for delete to authenticated
    using (exists (select 1 from public.events e where e.id = event_id and e.owner_id =
(select auth.uid())));
create policy "Guest answers" on public.event_invites for update to authenticated
    using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
revoke update on public.event_invites from authenticated, anon;
grant update (status, responded_at) on public.event_invites to authenticated; -- guests
can only change their answer

create or replace function public.my_invites()
returns table (id uuid, owner_id uuid, title text, location text, notes text, starts_at
timestamptz, ends_at timestamptz,
    all_day boolean, rrule text, exdates timestamptz[], my_status text, guests
jsonb)
language sql stable security definer set search_path = '' as $$
    select e.id, e.owner_id, e.title, e.location, e.notes, e.starts_at, e.ends_at, e.all_day,
e.rrule, e.exdates, i.status,
        (select coalesce(jsonb_agg(jsonb_build_object('user_id', g.user_id, 'status',
g.status)), '[]'::jsonb)
            from public.event_invites g where g.event_id = e.id)
    from public.event_invites i
    join public.events e on e.id = i.event_id
    where i.user_id = (select auth.uid())
        and public.is_friend(e.owner_id)
        and (e.rrule is not null or e.ends_at > now() - interval '7 days');
$$;
revoke execute on function public.my_invites() from public, anon;
grant execute on function public.my_invites() to authenticated;

create table public.broadcasts (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null default auth.uid() references public.profiles(id) on delete
cascade,
    message text not null check (char_length(btrim(message)) between 1 and 140),
    starts_at timestamptz not null default now(),
    ends_at timestamptz not null,
    created_at timestamptz not null default now(),
    check (ends_at > starts_at and ends_at <= starts_at + interval '7 days')
);
alter table public.broadcasts enable row level security;
create policy "Own or friends' broadcasts" on public.broadcasts for select to authenticated
    using (user_id = (select auth.uid()) or public.is_friend(user_id));

```

```

create policy "Post own broadcast" on public.broadcasts for insert to authenticated with
check (user_id = (select auth.uid()));
create policy "Edit own broadcast" on public.broadcasts for update to authenticated using
(user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
create policy "Delete own broadcast" on public.broadcasts for delete to authenticated using
(user_id = (select auth.uid()));

create table public.broadcast_replies (
  broadcast_id uuid not null references public.broadcasts(id) on delete cascade,
  user_id uuid not null default auth.uid() references public.profiles(id) on delete
cascade,
  reply text not null default 'in' check (reply in ('in','maybe')),
  created_at timestampz not null default now(),
  primary key (broadcast_id, user_id)
);
alter table public.broadcast_replies enable row level security;
create policy "See replies" on public.broadcast_replies for select to authenticated
  using (user_id = (select auth.uid()) or public.is_friend(user_id)
    or exists (select 1 from public.broadcasts b where b.id = broadcast_id and b.user_id
= (select auth.uid())));
create policy "Reply to friends' broadcasts" on public.broadcast_replies for insert to
authenticated
  with check (user_id = (select auth.uid())
    and exists (select 1 from public.broadcasts b where b.id = broadcast_id and
public.is_friend(b.user_id) and b.ends_at > now()));
create policy "Change own reply" on public.broadcast_replies for update to authenticated
  using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
create policy "Remove own reply" on public.broadcast_replies for delete to authenticated
using (user_id = (select auth.uid()));

alter table public.profiles add column status_text text check (char_length(status_text) <=
60);
alter table public.profiles add column status_until timestampz;
-- alter publication supabase_realtime add table public.event_invites, public.broadcasts,
public.broadcast_replies;

-- =====
-- Phase 10 (part 1): push notifications
-- =====
-- app_secret(n) / set_app_secret(n, v): service_role only. The push function creates its
VAPID key pair
-- on first run and keeps it in private.app_secrets (vapid_public, vapid_private), so the
private key never
-- leaves the server. vapid_public_key() returns the public half to signed-in users.

create table public.push_subscriptions (          -- one row per phone/browser that
allowed notifications
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid() references public.profiles(id) on delete
cascade,
  endpoint text not null unique check (endpoint like 'https://%'),
  p256dh text not null, auth text not null, device text,
  created_at timestampz not null default now(), last_ok_at timestampz
);
-- RLS: owner only

create table public.notify_settings (          -- RLS: owner only
  user_id uuid primary key default auth.uid() references public.profiles(id) on delete
cascade,

```

```

    friend_events boolean not null default true, plans boolean not null default true,
    broadcasts boolean not null default true, requests boolean not null default true,
    reminders boolean not null default true,
    default_reminder int, -- minutes before; null = off
    quiet_start time, quiet_end time, -- friend alerts held during these
hours
    muted uuid[] not null default '{}',
    updated_at timestampz not null default now()
);

alter table public.events add column remind_minutes int; -- null = use default, -1 = none

create table public.notifications ( -- inbox + push queue
    id bigint generated always as identity primary key,
    user_id uuid not null references public.profiles(id) on delete cascade,
    kind text not null check (kind in
('friend_event', 'invite', 'rsvp', 'broadcast', 'broadcast_reply', 'request', 'accepted', 'reminde
r', 'test')),
    actor uuid references public.profiles(id) on delete cascade,
    title text not null, body text, url text, ref text,
    count int not null default 1,
    created_at timestampz not null default now(),
    send_after timestampz not null default now(), -- friend_event waits 3 min so changes
are batched
    sent_at timestampz, pushed boolean, read_at timestampz
);
-- RLS: owner can read and delete; can only UPDATE read_at (column grant). Rows are only
written by triggers.
-- unique (user_id, ref) where kind = 'reminder' -> each reminder fires once

-- Triggers (SECURITY DEFINER, in schema private), all respecting notify_settings + muted
friends:
-- notify_on_activity friend added/changed an event -> each friend (batched per
friend for 3 minutes)
-- notify_on_invite invite -> guest; Going/Maybe/Can't -> host
-- notify_on_broadcast "up for something" -> each friend
-- notify_on_broadcast_reply "I'm in" -> poster
-- notify_on_friendship request -> addressee; accepted -> requester
-- cron 'klander-push' every minute: private.run_push() -> Edge Function push
-- makes due reminders (rrule-aware, time-zone safe), sends Web Push with VAPID, skips
quiet hours,
-- drops dead subscriptions (404/410), deletes notifications older than 30 days.

-- =====
-- Phase 7 (part 2): close friends, groups, countdowns, time polls
-- =====
create table public.close_friends ( -- your private list; RLS owner only,
friend must be a friend
    user_id uuid not null default auth.uid() references public.profiles(id) on delete
cascade,
    friend_id uuid not null references public.profiles(id) on delete cascade,
    created_at timestampz not null default now(),
    primary key (user_id, friend_id)
);
-- is_close_to_me(owner) -> am I on their list? events.visibility now allows 'close':
-- friend_events() shows close events in full to close friends and as 'Busy' to everyone
else;
-- log_event_activity() never puts a close event's title in the shared activity feed.

```

```

create table public.friend_groups (                                -- your own groups of friends; RLS
owner only
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references public.profiles(id) on delete
cascade,
  name text not null, members uuid[] not null default '{}', sort int not null default 0,
  created_at timestamptz not null default now()
);

alter table public.events add column countdown boolean not null default false;

create table public.polls (id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references public.profiles(id) on delete
cascade,
  title text not null, location text, note text, closes_at timestamptz,
  decided_option uuid, event_id uuid references public.events(id) on delete set null,
  created_at timestamptz not null default now());
create table public.poll_options (id uuid primary key default gen_random_uuid(),
  poll_id uuid not null references public.polls(id) on delete cascade,
  starts_at timestamptz not null, ends_at timestamptz not null, check (ends_at >
starts_at));
create table public.poll_invitees (poll_id uuid not null references public.polls(id) on
delete cascade,
  user_id uuid not null references public.profiles(id) on delete cascade, primary key
(poll_id, user_id));
create table public.poll_votes (option_id uuid not null references public.poll_options(id)
on delete cascade,
  user_id uuid not null default auth.uid() references public.profiles(id) on delete
cascade,
  vote text not null check (vote in ('yes','maybe','no')), updated_at timestamptz not null
default now(),
  primary key (option_id, user_id));
-- RLS: can_see_poll(p) = owner or invitee. Only the owner adds options/invitees (friends
only).
-- Invitees vote only for themselves, and only until the owner picks a time
(decided_option).
-- Triggers: notify_on_poll_invite -> invitee; notify_on_poll_vote -> owner (one per voter,
held 2 min).

-- =====
-- Phase 8: chat
-- =====
-- conversations (kind dm | group | event; dm_key "a:b" makes each pair have one DM;
event_id makes one chat per plan)
-- conversation_members (last_read_at for unread counts + "Seen", muted)
-- messages (text | image | event card | system; reply_to; soft delete via deleted_at;
edits only to body)
-- message_reactions (8 fixed emoji), blocks (owner only), reports (insert only, read in
the dashboard)
-- Storage bucket 'chat' (private, 5 MB, images only): path <conversation_id>/<uuid>.jpg,
readable/uploadable by members only.
-- RPCs (SECURITY DEFINER): start_dm(other) friends only, not blocked; create_group(title,
members) friends only, max 30;
-- event_thread(event) host + guests only; my_conversations() list with last message +
unread count.
-- RLS: is_member(conversation) gates everything. You can only send as yourself, never
'system' messages,

```

```

-- and not into a DM with someone who blocked you (or you blocked). Anyone in a group can
add their own friends.
-- Trigger notify_on_message: one notification per chat per person, updated with a running
count while unsent;
-- respects muted chats, the 'chat' setting, muted friends and blocks.

-- Chat photo cleanup (daily, cron 'klander-cleanup' 03:17 UTC -> Edge Function cleanup):
-- messages.image_removed_at; chat_photos_to_remove(lim) + mark_photos_removed(ids) are
service_role only.
-- Removes photos from deleted messages, photos older than 6 months, and uploads never sent
(older than a day).

-- =====
-- Phase 9: memories + My Week
-- =====
-- event_photos (event_id, user_id, path '<event_id>/<uuid>.jpg', caption, taken_at);
storage bucket 'memories' (private, 3 MB, jpeg/webp)
-- can_view_event(ev): host, invited (not declined), or a friend who can see the event's
details (friends / close, not hidden_from)
-- can_add_event_photo(ev): host, or a guest who said Going/Maybe
-- can_view_photo(id): your own, or you can see the event, or it's in a friend's posted My
Week you're allowed to see
-- week_recaps (user_id, week_start, slides jsonb, caption, visibility friends|close,
posted_at) unique per user+week;
-- hidden slides are saved WITHOUT photo_id/path so nobody can load a photo you hid.
-- week_reactions (one emoji per person per week). Triggers: notify_on_week (friends),
notify_on_week_reaction (owner).
-- cron 'klander-week-ready' Sundays 17:45 UTC: "Your week is ready" for anyone with photos
or 3+ events that week.
-- memory_orphans(): cleanup removes memory files whose photo row/event was deleted.

-- =====
-- Phase 10 (part 2): family link, shifts & sleep, morning brief
-- =====
-- family_feed_tokens (user_id pk, token, active): my_family_token(reset),
stop_family_link(), family_link_on().
-- The feed function serves these as: Friends events normally, Close/Busy-only as "Busy",
Private never, no notes.
-- categories.is_shift + categories.default_reminder; profiles.sleep_hours (default 7).
-- A sleep block follows each shift; friend_busy_v2() also returns is_shift + sleep_hours
so the finder respects friends' sleep.
-- notify_settings.morning_brief, brief_time, quiet_follow_sleep.
-- notifications.dedupe_key (unique) so reminders and briefs are only made once
-- (the earlier partial unique index couldn't be used by ON CONFLICT, so reminders were
silently not being created).
-- Phase 11: hangout stats. Only ever returns counts that involve the caller, and only with
friends.
create or replace function public.hangout_counts(p_from timestampz, p_to timestampz)
returns table (friend_id uuid, plans int, last_at timestampz)
language sql stable security definer set search_path = '' as $$
with me as (select (select auth.uid()) as id),
ev as (
select e.id, e.starts_at, e.owner_id
from public.events e
where e.starts_at >= p_from and e.starts_at < least(p_to, now())
and e.starts_at > now() - interval '400 days'
and e.source_id is null
),

```

```

people as (
  select ev.id, ev.starts_at, ev.owner_id as uid from ev
  union
  select ev.id, ev.starts_at, i.user_id from ev join public.event_invites i on i.event_id
= ev.id and i.status = 'going'
),
mine as (select p.id from people p, me where p.uid = me.id)
select p.uid, count(distinct p.id)::int, max(p.starts_at)
from people p join mine m on m.id = p.id, me
where p.uid <> me.id
  and exists (select 1 from public.friendships f where f.status = 'accepted'
              and ((f.requester = me.id and f.addressee = p.uid) or (f.addressee = me.id
and f.requester = p.uid)))
  and not exists (select 1 from public.blocks b where (b.user_id = me.id and b.blocked_id
= p.uid) or (b.user_id = p.uid and b.blocked_id = me.id))
group by p.uid
$$;
revoke execute on function public.hangout_counts(timestamptz, timestamptz) from public,
anon;
grant execute on function public.hangout_counts(timestamptz, timestamptz) to authenticated;

```

Appendix B: Key code excerpts

B1. Expanding repeating events (src/lib/dates.js)

Uses "floating time" so repeats stay at the same local time across clock changes.

```
const toFloating = d => new Date(Date.UTC(d.getFullYear(), d.getMonth(), d.getDate(),
d.getHours(), d.getMinutes(), d.getSeconds()))
const fromFloating = d => new Date(d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate(),
d.getUTCHours(), d.getUTCMinutes(), d.getUTCSeconds())

/** Expand stored events into concrete occurrences overlapping [from, to). */
export function expandEvents(events, from, to) {
  const out = []
  for (const ev of events) {
    const start = new Date(ev.starts_at)
    const end = new Date(ev.ends_at)
    const durMin = differenceInMinutes(end, start)
    if (!ev.rrule) {
      if (start < to && end > from) out.push({ ...ev, start, end, key: ev.id })
      continue
    }
    let opts
    try { opts = RRule.parseString(ev.rrule) } catch { continue }
    // UNTIL is written as local wall-clock time with a Z suffix (see buildRRule), so it is
    already "floating".
    const rule = new RRule({ ...opts, dtstart: toFloating(start) })
    const ex = new Set((ev.exdates || []).map(x => new Date(x).getTime()))
    const occ = rule.between(toFloating(addMinutes(from, -durMin)), toFloating(to), true)
    for (const f of occ) {
      const s = fromFloating(f)
      if (ex.has(s.getTime())) continue
      const e = addMinutes(s, durMin)
      if (s < to && e > from) out.push({ ...ev, start: s, end: e, key: `${ev.id}:$
{s.getTime()}`, occurrence: s })
    }
  }
  return out.sort((a, b) => a.start - b.start || b.end - a.end)
}
```

B2. Overlap layout (src/lib/dates.js)

```
/** Lay out timed events for one day column: returns [{ev, top, height, left, width}] in
% / minutes. */
export function layoutDay(occurrences, day) {
  const dayStart = startOfDay(day)
  const dayEnd = addDays(dayStart, 1)
  const items = occurrences
    .filter(o => !o.all_day && o.start < dayEnd && o.end > dayStart)
    .map(o => {
      const s = o.start < dayStart ? dayStart : o.start
      const e = o.end > dayEnd ? dayEnd : o.end
      const top = differenceInMinutes(s, dayStart)
      return { ev: o, top, bottom: Math.max(top + 20, differenceInMinutes(e, dayStart)) }
    })
}
```

```

        .sort((a, b) => a.top - b.top || b.bottom - a.bottom)

// cluster overlapping events, then assign columns greedily inside each cluster
const result = []
let cluster = [], clusterEnd = -1
const flush = () => {
    const cols = []
    for (const it of cluster) {
        let c = cols.findIndex(end => end <= it.top)
        if (c === -1) { c = cols.length; cols.push(0) }
        cols[c] = it.bottom
        it.col = c
    }
    for (const it of cluster) result.push({ ...it, cols: cols.length })
    cluster = []
}
for (const it of items) {
    if (it.top >= clusterEnd && cluster.length) flush()
    cluster.push(it)
    clusterEnd = Math.max(clusterEnd, it.bottom)
}
if (cluster.length) flush()
return result
}

```

B3. Friends' events with privacy applied (database function)

```

-- Friends' events with privacy applied: private never returned, busy shows only "Busy",
notes never shared
create function public.friend_events()
returns table (id uuid, owner_id uuid, title text, location text, starts_at timestampz,
ends_at timestampz,
               all_day boolean, visibility text, rrule text, exdates timestampz[])
language sql stable security definer set search_path = '' as $$
    select e.id, e.owner_id,
           case when e.visibility = 'busy' then 'Busy' else e.title end,
           case when e.visibility = 'busy' then null else e.location end,
           e.starts_at, e.ends_at, e.all_day, e.visibility, e.rrule, e.exdates
    from public.events e
    join public.friendships f on f.status = 'accepted'
    and ((f.requester = (select auth.uid()) and f.addressee = e.owner_id)
        or (f.addressee = (select auth.uid()) and f.requester = e.owner_id))
    where e.visibility <> 'private';
$$;

```

B4. Activity trigger (database)

Writes a privacy-safe activity row whenever an event changes. Imported events are skipped so a timetable sync doesn't flood the feed.

```

create function public.log_event_activity() returns trigger
language plpgsql security definer set search_path = '' as $$
declare v_old text; v_new text;
begin
    if coalesce(new.source_id, old.source_id) is not null then return null; end if;

```

```

v_old := case when tg_op in ('UPDATE','DELETE') then old.visibility end;
v_new := case when tg_op in ('INSERT','UPDATE') then new.visibility end;
if tg_op = 'INSERT' then
  if v_new = 'private' then return null; end if;
  insert into public.activity(actor, verb, title, starts_at)
    values (new.owner_id, 'added', case when v_new = 'busy' then null else new.title end,
new.starts_at);
elsif tg_op = 'UPDATE' then
  if v_old = 'private' and v_new = 'private' then return null; end if;
  if v_new = 'private' then
    insert into public.activity(actor, verb) values (new.owner_id, 'removed');
  else
    insert into public.activity(actor, verb, title, starts_at)
      values (new.owner_id, case when v_old = 'private' then 'added' else 'changed' end,
        case when v_new = 'busy' then null else new.title end, new.starts_at);
  end if;
else
  if v_old = 'private' then return null; end if;
  insert into public.activity(actor, verb) values (old.owner_id, 'removed');
end if;
return null;
end; $$;

```

B5. Live updates (src/lib/friends.js)

```

// Live updates: friends' changes and friend requests arrive straight away
useEffect(() => {
  let timer = null
  const soon = () => { clearTimeout(timer); timer = setTimeout(refresh, 400) }
  const channel = supabase.channel(`klander-live-${uid}`)
    .on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'activity' },
soon)
    .on('postgres_changes', { event: '*', schema: 'public', table: 'friendships' }, soon)
    .subscribe()
  return () => { clearTimeout(timer); supabase.removeChannel(channel) }
}, [uid, refresh])

```

B6. Reading calendar time zones (supabase/functions/_shared/ics.js)

```

/** ical.js Time -> { date: Date, allDay: boolean } */
function timeToInstant(ICAL, t, tz) {
  if (t.isDate) return { date: zonedToUtc(t.year, t.month, t.day, 0, 0, 0, tz), allDay:
true }
  const zone = t.zone
  const known = zone && (zone === ICAL.Timezone.utcTimezone || (zone.tzid && zone.tzid !==
'floating' && zone.component))
  if (known) return { date: t.toJSDate(), allDay: false }
  // Floating time, or a TZID the file didn't define (common with Outlook): try it as an
IANA zone, else the user's zone
  const tzid = zone?.tzid && zone.tzid !== 'floating' ? zone.tzid : null
  let useTz = tz
  if (tzid) { try { new Intl.DateTimeFormat('en-GB', { timeZone: tzid }); useTz = tzid }
catch { /* not IANA */ } }
  return { date: zonedToUtc(t.year, t.month, t.day, t.hour, t.minute, t.second, useTz),
allDay: false }
}

```

```
}
```

B7. Smart add prompt rules (supabase/functions/smart-add)

```
function instructions(now: string, tz: string) {
  return `You turn messages, posters, screenshots, tickets, timetables and rotas into
calendar events for a UK user.
Right now it is ${now} (time zone ${tz}). Dates are UK style (day before month).
Rules:
- Resolve relative dates ("thurs", "tomorrow", "next week Friday") to the next matching
date on or after today.
- Times: 24-hour HH:MM. "7" or "7pm" for an evening activity is 19:00. If no end time,
leave end_time null.
- If there is no time at all, set all_day true and start_time null.
- A timetable or rota: one event per distinct class/shift. If it is a weekly timetable, use
repeat "weekly" with repeat_days set, and the date of its next occurrence. Shifts that end
after midnight need end_date set to the next day.
- Group chats: only make events for plans that are actually proposed or agreed, not past
ones.
- Titles: short and natural, no emoji, no dates in the title.
- Never invent details that are not shown. If nothing looks like an event, return an empty
list and explain in "message".`
}
```

B8. Turning an AI draft into an event (src/lib/smart.js)

```
/** AI draft -> the shape saveEvent() expects. */
export function draftToEvent(d, { category_id = null, visibility = 'friends' } = {}) {
  let s, e
  if (d.all_day) {
    s = startOfDay(at(d.date))
    e = addDays(startOfDay(at(d.end_date && d.end_date >= d.date ? d.end_date : d.date)),
1)
  } else {
    s = at(d.date, d.start_time)
    if (d.end_time) {
      e = at(d.end_date || d.date, d.end_time)
      if (e <= s) e = addDays(e, 1) // e.g. 23:30 -> 00:30 night shift
    } else e = addMinutes(s, 60)
  }
  let rrule = null
  if (d.repeat === 'weekly' && d.repeat_days?.length) {
    rrule = `FREQ=WEEKLY;BYDAY=${d.repeat_days.join(',')}`
    if (d.repeat_until) rrule += `;UNTIL=${d.repeat_until.replace(/-/g, '')}T235959Z`
  } else if (d.repeat && d.repeat !== 'none') {
    rrule = buildRRule(d.repeat, s, d.repeat_until || null)
  }
  return {
    title: d.title, all_day: !!d.all_day, starts_at: s.toISOString(), ends_at:
e.toISOString(),
    location: d.location || '', notes: d.notes || '', rrule, exdates: [], category_id,
visibility
  }
}
```

B9. Free-time finder (src/lib/freetime.js)

```
export function findTimes(busyBy, people, { days = 7, duration = 60, startHour = 9, endHour
= 22, now = new Date() } = {}) {
  const all = [], allBut = []
  const first = startOfDay(now)
  for (let i = 0; i < days; i++) {
    const day = addDays(first, i)
    let ws = new Date(day); ws.setHours(startHour, 0, 0, 0)
    let we = new Date(day); if (endHour >= 24) { we = addDays(day, 1) } else
we.setHours(endHour, 0, 0, 0)
    if (we <= now) continue
    if (ws < now) ws = roundUp(now)
    if (addMinutes(ws, duration) > we) continue
    const gapsFor = ids => {
      const busy = merge(ids.flatMap(id => busyBy[id] || []))
      return freeGaps(busy, ws, we).filter(g => (g.e - g.s) / 60000 >= duration)
    }
    for (const g of gapsFor(people)) all.push({ ...g, day })
    if (people.length > 2) {
      for (const missing of people) {
        const rest = people.filter(p => p !== missing)
        for (const g of gapsFor(rest)) {
          // only worth showing if it isn't already an everyone-free window
          if (!all.some(a => a.s <= g.s && a.e >= g.e)) allBut.push({ ...g, day, missing })
        }
      }
    }
  }
  allBut.sort((a, b) => a.s - b.s)
  return { all, allBut: allBut.slice(0, 12) }
}
```

B10. Sleep after shifts (src/lib/sleep.js)

```
/** Sleep blocks after shifts: one per shift occurrence, `hours` long. */
export function sleepBlocks(occurrences, isShift, hours = 7) {
  if (!hours) return []
  return occurrences
    .filter(o => !o.all_day && isShift(o))
    .map(o => ({
      id: `sleep-${o.key}`, key: `sleep-${o.key}`, title: 'Sleep', sleep: true, all_day:
false,
      start: o.end, end: new Date(o.end.getTime() + hours * 3600e3), visibility: 'busy'
    )))
}
```

B11. Streaks (src/lib/insights.js)

```
export function streaks({ events, categories, now = new Date(), maxWeeks = 52 }) {
  const thisWeek = startOfWeek(now, WEEK_OPTS)
  const from = addWeeks(thisWeek, -maxWeeks)
  const occ = expandEvents(events.filter(e => e.category_id), from, addWeeks(thisWeek, 1))
  const out = []
  for (const c of categories) {
```

```

    if (c.is_shift) continue
    const weeks = new Set(occ.filter(o => o.category_id === c.id && o.start <= now).map(o
=> +startOfWeek(o.start, WEEK_OPTS)))
    const doneThisWeek = weeks.has(+thisWeek)
    let w = doneThisWeek ? thisWeek : addWeeks(thisWeek, -1)
    let n = 0
    while (weeks.has(+w)) { n++; w = addWeeks(w, -1) }
    // longest run in the window
    let best = 0, run = 0
    for (let i = maxWeeks; i >= 0; i--) { if (weeks.has(+addWeeks(thisWeek, -i))) { run++;
best = Math.max(best, run) } else run = 0 }
    if (n >= 2 || best >= 3) out.push({ id: c.id, name: c.name, colour: c.colour, weeks: n,
doneThisWeek, best })
  }
  return out.sort((a, b) => b.weeks - a.weeks || b.best - a.best)
}

```

B12. Hangout counts (database function)

```

create or replace function public.hangout_counts(p_from timestampz, p_to timestampz)
returns table (friend_id uuid, plans int, last_at timestampz)
language sql stable security definer set search_path = '' as $$
with me as (select (select auth.uid()) as id),
ev as (
  select e.id, e.starts_at, e.owner_id
  from public.events e
  where e.starts_at >= p_from and e.starts_at < least(p_to, now())
    and e.starts_at > now() - interval '400 days'
    and e.source_id is null
),
people as (
  select ev.id, ev.starts_at, ev.owner_id as uid from ev
  union
  select ev.id, ev.starts_at, i.user_id from ev join public.event_invites i on i.event_id
= ev.id and i.status = 'going'
),
mine as (select p.id from people p, me where p.uid = me.id)
select p.uid, count(distinct p.id)::int, max(p.starts_at)
from people p join mine m on m.id = p.id, me
where p.uid <> me.id
  and exists (select 1 from public.friendships f where f.status = 'accepted'
    and ((f.requester = me.id and f.addressee = p.uid) or (f.addressee = me.id
and f.requester = p.uid)))
  and not exists (select 1 from public.blocks b where (b.user_id = me.id and b.blocked_id
= p.uid) or (b.user_id = p.uid and b.blocked_id = me.id))
group by p.uid
$$;

```

Appendix C: Glossary

Term	Meaning
PWA	Progressive Web App: a website that can be installed to the Home Screen and work offline
Supabase	A hosted backend: Postgres database, authentication, storage, realtime and serverless functions
RLS	Row Level Security: database rules deciding which rows each user can read or change
SECURITY DEFINER	A database function that runs with its owner's rights; used carefully to return filtered data
Edge Function	Server-side code (TypeScript on Deno) that runs on Supabase, used for anything needing secrets
JWT	JSON Web Token: the signed token that proves who you are on each request
OAuth	The standard behind "Continue with Google"
.ics / iCalendar	The standard calendar file format used by Apple, Google and Outlook; webcal:// is a link to one
RRULE	A repeat rule in iCalendar format, e.g. <code>FREQ=WEEKLY;BYDAY=MO,TH</code>
EXDATE	A skipped date in a repeating event
pg_cron / pg_net	Postgres extensions for scheduled jobs and making web requests from the database
Realtime	Supabase's service that pushes database changes to open apps
Gemini	Google's AI model, used to read text and photos for smart add
BST / GMT	British Summer Time (UTC+1) and Greenwich Mean Time (UTC+0); clocks change in March and October
Web Push / VAPID	The standard way a website sends lock-screen notifications; VAPID keys prove the notifications come from Klander
Service worker	A small script the phone keeps running for the app: offline caching and receiving notifications
dedupe key	A unique label on a notification so the same reminder can never be created twice
k-means	A clustering method used to pick the main colours out of a photo for a theme